



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka Bengaluru - 560064



**SUMMER-TERM TIMETABLE GENERATION
A PROJECT REPORT**

Submitted by

KEERTHI D V- 20221COM0100

VIDYA S- 20221COM0108

NISHMITHA K G- 20221COM0111

Under the guidance of,

Dr. Sandeep Albert Mathias

**BACHELOR OF TECHNOLOGY
IN
COMPUTER ENGINEERING**

PRESIDENCY UNIVERSITY

BENGALURU

DECEMBER 2025



PRESIDENCY UNIVERSITY

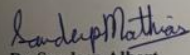
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka Bengaluru - 560064



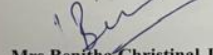
**PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND
ENGINEERING**

BONAFIDE CERTIFICATE

Certified that this report "SUMMER-TERM TIMETABLE GENERATION" is a bonafide work of "KEERTHI D V(20221COM0100), VIDYA S(20221COM0108), NISHMITHA K G(20221COM0111)", who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER ENGINEERING during 2025-26.


Dr. Sandeep Albert Mathias

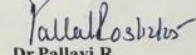
Project Guide
PSCS
Presidency University


Mrs. Benitha Christinal J

Program Project
Coordinator
PSCS
Presidency University


Dr. Sampath A K

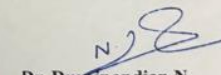
Dr. Geetha A
School Project
Coordinators
PSCS
Presidency University


Dr. Pallavi R

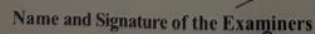
Head of the Department
PSCS
Presidency University


Dr. Shakkeera L

Associate Dean
PSCS
Presidency University

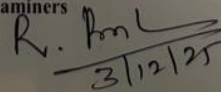

Dr. Duraisundar N

Dean
PSCS & PSIS
Presidency University


Name and Signature of the Examiners

1) Dr R Balakrishna

2) Mr. Sakthi S


3/12/25

PRESIDENCY UNIVERSITY
PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND
ENGINEERING
DECLARATION

We the students of final year B.Tech in COMPUTER ENGINEERING at Presidency University, Bengaluru, named KEERTHI D V, VIDYA S, and NISHMITHA K G hereby declare that the project work titled "**SUMMER-TERM TIMETABLE GENERATION**" has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER ENGINEERING during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

KEERTHI D V	USN: 20221COM0100	<i>Keerthi</i>
VIDYA S	USN: 20221COM0108	<i>Vidya S</i>
NISHMITHA K G	USN: 20221COM0111	<i>Nishmitha</i>

PLACE: BENGALURU

DATE: 03-December 2025

ACKNOWLEDGEMENT

For completing this project work, We/I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide **Dr.Sandeep Albert Mathias,Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, for moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Pallavi R, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, **Dr. Shakkeera L**, Associate Dean, Presidency School of Computer Science and Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K, and Dr. Geetha A**, PSCS Project Coordinators, **Mrs. Benitha Christinal J**, Program Project Coordinator, Presidency School of Computer Science and Engineering, for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

KEERTHI D V

VIDYA S

NISHMITHA K G



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka| Bengaluru – 560064



SUMMER-TERM TIMETABLE GENERATION

A PROJECT REPORT

Submitted by

KEERTHI D V- 20221COM0100

VIDYA S- 20221COM0108

NISHMITHA K G- 20221COM0111

Under the guidance of,

Dr. Sandeep Albert Mathias

BACHELOR OF TECHNOLOGY

IN

COMPUTER ENGINEERING

PRESIDENCY UNIVERSITY

BENGALURU

DECEMBER 2025



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka| Bengaluru – 560064



PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report “SUMMER-TERM TIMETABLE GENERATION” is a bonafide work of “KEERTHI D V(20221COM0100),VIDYA S(20221COM0108),NISHMITHA K G(20221COM0111)”, who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER ENGINEERING during 2025-26.

**Dr. Sandeep Albert
Mathias**
Project Guide
PSCS
Presidency University

Mrs.Benitha Christinal J
Program Project
Coordinator
PSCS
Presidency University

**Dr. Sampath A K
Dr. Geetha A**
School Project
Coordinators
PSCS
Presidency University

Dr.Pallavi R
Head of the Department
PSCS
Presidency University

Dr. Shakkeera L
Associate Dean
PSCS
Presidency University

Dr. Duraipandian N
Dean
PSCS & PSIS
Presidency University

Name and Signature of the Examiners

1)Dr R Balakrishna

2)Mr.Sakthi S

PRESIDENCY UNIVERSITY
PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND
ENGINEERING
DECLARATION

We the students of final year B.Tech in COMPUTER ENGINEERING at Presidency University, Bengaluru, named KEERTHI D V, VIDYA S, and NISHMITHA K G hereby declare that the project work titled “**SUMMER-TERM TIMETABLE GENERATION**” has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER ENGINEERING during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

KEERTHI D V USN: 20221COM0100
VIDYA S USN: 20221COM0108
NISHMITHA K G USN: 20221COM0111

PLACE: BENGALURU

DATE: 03-December 2025

ACKNOWLEDGEMENT

For completing this project work, We/I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide **Dr.Sandeep Albert Mathias,Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, for moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Pallavi R, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, **Dr. Shakkeera L**, Associate Dean, Presidency School of Computer Science and Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K**, and **Dr. Geetha A**, PSCS Project Coordinators, **Mrs. Benitha Christinal J**, **Program Project Coordinator**, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

KEERTHI D V
VIDYA S
NISHMITHA K G

Abstract

Efficient timetable generation is a recurring challenge in academic institutions, especially during summer terms when course offerings, student enrollments, and resource availability differ significantly from regular semesters. Traditional manual scheduling often leads to clashes, uneven resource distribution, limited flexibility, and high administrative effort. These issues become more prominent when multiple departments, diverse course combinations, and variable registration patterns must be handled simultaneously. To address these challenges, this project focuses on automating the summer-term timetable creation process by analyzing student registration data and applying systematic scheduling logic.

The proposed approach integrates data preprocessing, graph-based conflict identification, course grouping, section creation, and automated resource allocation into a unified pipeline. The system begins by cleaning the registration dataset, removing MOOC courses and courses with fewer than the required threshold of valid registrations. A conflict graph is then constructed, where each course is treated as a node and edges represent clashes based on shared student enrollments. A greedy graph-coloring algorithm is applied to organize courses into non-overlapping groups, ensuring that no student faces timetable conflicts. These groups are mapped to predefined time slots across an academic week. To handle high-enrollment courses, the model automatically splits them into multiple sections and assigns two faculty members and an available room to each section using round-robin allocation. The system ultimately generates an expanded, conflict-free timetable that incorporates sections, faculty distribution, and room utilization.

The results demonstrate that the automated method successfully produces a comprehensive and clash-free summer-term timetable that is scalable, consistent, and efficient. The output ensures that student conflicts are eliminated, faculty assignments are balanced, and resources such as rooms are utilized optimally. The model reduces manual workload, improves scheduling accuracy, and provides a flexible framework that can be extended for larger datasets, additional constraints, and real-world institutional requirements. This project thus offers a practical and reliable solution to the complexities of academic timetable generation.

Table of Content

Sl. No.	Title	Page No.
	Declaration	I
	Acknowledgement	II
	Abstract	III
	List of Figures	VI
	List of Tables	VII
	Abbreviations	VIII
1.	Introduction 1.1 Background 1.2 Statistics of project 1.3 Prior existing technologies 1.4 Proposed approach 1.5 Objectives 1.6 SDGs 1.7 Overview of project report	1-11
2.	Literature review	12-15
3.	Methodology	16-25
4.	Project management 4.1 Project timeline 4.2 Risk analysis 4.3 Project budget	26-29
5.	Analysis and Design 5.1 Requirements 5.2 Block Diagram 5.3 System Flow Chart 5.4 Choosing Software Components 5.5 Designing units 5.6 Standards 5.7 Other Design	30-48
6.	Software and Simulation	49-55

	6.1 Software development tools 6.2 Software code 6.3 Simulation	
7.	Evaluation and Results 7.1 Test points 7.2 Test plan 7.3 Test result 7.4 Insights	56-61
8.	Social, Legal, Ethical, Sustainability and Safety Aspects 8.1 Social aspects 8.2 Legal aspects 8.3 Ethical aspects 8.4 Sustainability aspects 8.5 Safety aspects	62-66
9.	Conclusion	67
	References	68-69
	Base Paper	70
	Appendix	71-72

List of Figures

Figure	Caption	Page no
Fig 1.6.1	Sustainable development goals [1]	10
Fig 3.1	The V-Model methodology	16
Fig 3.2	Another example of the V-Model methodology	16
Fig 3.3	W-Model Methodology Context	20
Fig 3.4	DevOps methodology	21
Fig 3.5	Onion methodology	22
Fig 3.6	SDLC phases	22
Fig 3.7	Summary of various methodologies	23
Fig 3.8	Some more methodologies	21
Fig 3.9	Summary of project breakdown to task	22
Fig 5.1	Functional Block Diagram	35
Fig 5.2	System Flow Chart	36

List of Tables

Table	Caption	Page no
Table 2.1	Summary of Literature reviews	15
Table 4.1	Project Planning Timeline	26
Table 4.1.2	Project Implementation Timeline	27
Table 4.1.3	Testing & Report Timeline	28
Table 4.2	PESTEL Analysis	28
Table 4.3	Project Budget	29
Table 5.1	Summarizing requirements	33
Table 5.4	Choosing Software Components	36
Table 7.2	Test Plan	58

Abbreviations

Abbreviation	Full Form
SDG	Sustainable Development Goals
DPDPA	Digital Personal Data Protection Act
GDPR	General Data Protection Regulation
IEEE	Institute of Electrical and Electronics Engineers
ISO	International Organization for Standardization
ICT	Information and Communication Technology
CSV	Comma Separated Values
PDF	Portable Document Format
AI	Artificial Intelligence (<i>mentioned in referenced works, not as a main system element</i>)
GA	Genetic Algorithm (<i>used in literature survey papers</i>)
ICT	Information and Communication Technology (<i>if referred in intro/methodology</i>)
ICACCS / ICCES / ICECA / Confluence	Names of IEEE Conferences

Chapter 1

Introduction

Timetable generation is one of the core administrative tasks in any academic institution, ensuring that classes, faculty, and physical resources are systematically organized. While regular semester timetables are generally planned well in advance, summer-term timetables—often offering a reduced but high-demand set of courses—pose unique scheduling challenges. These challenges arise from fluctuating student registrations, overlapping course requirements, limited faculty availability, and resource constraints. Traditional manual scheduling methods commonly used in many universities are time-consuming, prone to error, and difficult to scale, especially when handling large datasets and tight academic timelines. As institutions move toward digitalization and automation, the need for an efficient summer-term timetable generation mechanism becomes increasingly significant.

The aim of this project is to design and develop an automated system that generates a **conflict-free, faculty-balanced, and resource-optimized summer-term timetable** using student registration data. The project focuses on identifying course collisions, grouping non-conflicting subjects, creating sections for high-enrollment courses, and systematically assigning rooms and faculty members. By applying logical scheduling algorithms and constraint-based allocation, the system ensures that no student experiences timetable clashes while maintaining a balanced distribution of academic resources.

The need for this project is supported by the scale and patterns of student registrations. In typical summer terms, students enroll selectively—primarily in backlog or fast-track courses—resulting in uneven enrollment counts across subjects. Some courses attract fewer than 21 registrations and are not viable to conduct, while others exceed 60 students and must be split into multiple sections. These irregular patterns demand automated preprocessing, filtering, and scheduling to avoid human oversight. Without automation, institutions face challenges such as overlapping classes for the same student, uneven faculty workload distribution, and inefficient utilization of classrooms. Furthermore, with rising student populations and diversified course combinations, manual timetable creation becomes impractical and error-prone.

Prior technologies in timetable generation include **basic spreadsheet-based scheduling, rule-based academic planners**, and **off-the-shelf timetable software** such as aSc Timetables and

Untis. While these tools offer structured interfaces, they often function as closed systems and require manual constraint input, making them less adaptable for custom academic workflows or dynamic datasets. Recent research has explored **graph coloring**, **genetic algorithms**, and **heuristic optimization techniques** for timetable generation; however, these methods require significant computational setup and are not always suitable for small-to-medium institutions with specific scheduling rules. The approach used in this project—combining data preprocessing, conflict graph construction, and greedy graph-coloring—provides a practical and lightweight solution tailored to the institution’s needs while ensuring accuracy, flexibility, and reduced administrative workload.

1.1 Background

Timetable generation is a foundational administrative task in educational institutions, ensuring the structured allocation of classes, faculty, and physical resources within a constrained academic week. Traditionally, timetable preparation has been carried out manually using spreadsheets, charts, or ad-hoc tools, a process that becomes increasingly inefficient as the number of courses, students, and constraints grows (Carter & Laporte, 1998). Manual scheduling also introduces a high probability of human error, especially when handling diverse course combinations, large student enrollment datasets, and complex constraints such as faculty availability, classroom capacity, and course overlap restrictions.

In summer terms, the complexity intensifies because students typically enroll selectively—mostly for backlog clearance, repeat attempts, or fast-track progression—resulting in highly uneven registration patterns. Some courses attract very low enrollment, whereas others exceed classroom capacities and require multiple sections. These dynamic registration patterns demand automated preprocessing, conflict detection, and structured scheduling, which manual techniques fail to deliver efficiently. Existing research highlights that timetable scheduling problems fall under NP-Complete combinatorial optimization, meaning the complexity increases exponentially with data size (Lewis, 2008). Therefore, computational approaches are essential for generating feasible, conflict-free timetables within practical time constraints.

To address these challenges, recent systems and academic studies have incorporated algorithmic and data-driven solutions such as graph-coloring-based conflict resolution, heuristic optimization, and resource-aware scheduling. Graph-based models in particular have proven effective in modeling course conflicts by representing each course as a node and drawing edges between courses that share students (Tripathy & Tripathy, 2017). Such models

reduce scheduling complexity by grouping non-conflicting subjects into distinct time slots, ensuring that no student faces a timetable clash. Many modern timetable software tools, including aSc Timetables and Untis, also implement variations of these techniques, demonstrating the practical value of algorithmic scheduling (aSc Timetables, 2023).

This project builds on these established concepts by implementing a structured pipeline that performs data preprocessing, conflict graph generation, greedy graph coloring, section creation for high-enrollment courses, and automated assignment of faculty and rooms. By leveraging these computational methods, the project aims to deliver a scalable, accurate, and institution-friendly solution for generating efficient summer-term timetables.

1.2 Statistics

Higher education institutions in India handle extremely large and diverse student populations, making academic scheduling a significant logistical challenge. According to the *All India Survey on Higher Education (AISHE) 2023*, India currently enrolls over **4.33 crore** students in higher education, with engineering and technology disciplines alone accounting for nearly **36 lakh** enrollments (Ministry of Education, 2023). Universities and autonomous institutions frequently conduct **supplementary or summer terms** to support students who need to clear backlogs, improve grades, or fast-track degree progress. These terms consistently show **uneven registration patterns**, where some courses attract fewer than 20 students while others exceed 60–100 depending on institutional capacity and academic demand.

A 2022 study by the Karnataka State Higher Education Council (KSHEC) reported that nearly **18–22%** of engineering undergraduates in Karnataka enroll in summer-term or supplementary courses each year, primarily due to backlog clearance requirements (KSHEC, 2022). At the same time, course enrollment distribution is highly skewed—some subjects in core semesters record **less than 15** registrations, while popular or repeated courses often see **more than 70** enrollments. This variation directly impacts classroom allocation, faculty workload, section creation, and scheduling efficiency.

Such statistical trends highlight the **need for automated systems** to process registration data and generate conflict-free timetables. When scheduling is handled manually, administrators must evaluate hundreds of registration combinations, determine which courses should be offered, create sections for high-enrollment subjects, and avoid timetable clashes for students repeating multiple courses. Manual planning becomes even more challenging during summer terms, where compressed academic calendars demand faster decision-making. Studies

in academic scheduling clearly emphasize that manual or semi-manual scheduling leads to **higher error rates, inconsistent resource utilization, and significant delays** in academic planning (Schaerf, 1999).

Given the rising student numbers, irregular course-wise registrations, and increasing academic pressure to provide efficient supplementary terms, automated timetable generation has become a **critical institutional requirement**. The ability to analyze registrations, identify conflicts, group compatible subjects, and assign faculty and rooms algorithmically not only reduces administrative effort but also ensures fairness, transparency, and operational efficiency in academic scheduling.

1.3 Prior existing technologies

Timetable generation has been an active area of research and software development for several decades, leading to the emergence of multiple technologies and algorithmic approaches. Early systems predominantly relied on **manual and semi-automated scheduling tools**, such as spreadsheet-based planners or simple rule-based desktop applications. These methods required administrators to manually input constraints, verify clashes, and repeatedly adjust schedules, making them inefficient and prone to human error when dealing with large student populations (Carter & Laporte, 1998).

With advances in algorithmic research, several **computational models** were developed to automate academic scheduling. Among these, **graph-coloring techniques** became widely used due to their effectiveness in modeling course conflicts. In this approach, each course is represented as a node and an edge is drawn between two nodes if the courses share common students. Assigning colors to nodes represents assigning slots, with the graph-coloring algorithm ensuring that no two conflicting courses share the same time slot. This concept forms the foundation of many modern timetable-generation systems (Tripathy & Tripathy, 2017).

Another major class of solutions involves **heuristic and metaheuristic algorithms**, including **genetic algorithms (GA), simulated annealing, tabu search, and constraint satisfaction techniques**. These optimization-based methods search for near-optimal timetable configurations by exploring large solution spaces and adjusting schedules based on fitness evaluations. Lewis (2008) and Schaerf (1999) highlight that these approaches are particularly useful for large universities with complex constraints such as faculty availability, room capacities, and multi-section courses. However, they also come with high computational

overhead and require detailed parameter tuning, making them less accessible for institutions with limited technical resources.

In addition to algorithmic research, several **commercial and institutional scheduling tools** have emerged. Notable examples include **aSc Timetables**, **Untis**, and **FET Timetabling**, which provide graphical interfaces for generating and visualizing timetables. These tools implement a combination of rule-based engines and algorithmic scheduling techniques. For instance, aSc Timetables uses constraint-based optimization and automated conflict detection to help institutions generate practical timetables with minimal manual intervention (aSc Timetables, 2023). While these tools are robust and widely adopted, they often operate as closed-source systems, limiting customization for institution-specific workflows such as summer-term scheduling, backlog handling, or dynamic section creation based on student registrations.

In recent years, institutions have also explored **custom automation scripts** using programming languages like Python, integrating data preprocessing, algorithmic conflict detection, and resource allocation. Such solutions are more adaptable to institution-specific requirements, such as filtering courses with low enrollment, splitting high-enrollment subjects into multiple sections, and assigning faculty and rooms programmatically. This project builds upon these modern, flexible approaches by leveraging conflict graphs, greedy graph coloring, and automated section and resource allocation to produce a scalable and efficient timetable-generation system.

1.4 Proposed approach

- Aim of project

The primary aim of this project is to design and implement an automated system for generating a **conflict-free, faculty-balanced, and resource-optimized summer-term timetable** using student registration data. The system intends to eliminate manual scheduling errors, reduce administrative workload, and ensure that students can attend all their selected courses without any overlap in timings.

- Motivation

Summer-term scheduling presents unique challenges in higher education institutions due to irregular registration patterns, backlog courses, and limited resource availability. Traditional manual scheduling—often done using spreadsheets or ad-hoc planning—struggles to accommodate these variations efficiently. Conflicts between course

timings, uneven faculty assignment, and inefficient room usage frequently arise when schedules are created manually.

The need for automation becomes even more critical when student enrollments vary significantly across subjects. Courses with fewer than 21 registrations may not be offered, while high-demand courses require multiple sections to manage capacity. Motivated by these real-world challenges, this project adopts algorithmic methods to generate accurate and scalable timetables that adapt to dynamic enrollment data.

- **Proposed approach**

The proposed solution uses a structured, multi-stage computational workflow as follows:

- 1. Data Preprocessing**

- Read student registration data from the dataset.
- Remove MOOC courses and courses with fewer than 21 valid registrations.
- Construct mappings such as roll-to-courses and course-to-student lists.

- 2. Conflict Graph Construction**

- Treat each course as a node in a graph.
- Add edges between courses that share at least one student.
- This graph visually represents all course clashes.

- 3. Greedy Graph Coloring**

- Apply a greedy coloring algorithm to group non-conflicting courses.
- Each color corresponds to a unique time slot.
- Ensures that no student faces overlapping classes.

- 4. Time Slot Allocation**

- Map each colored group to predefined slots (9–11 AM, 11–1 PM, 2–4 PM).
- Spread groups across days to avoid resource congestion.

- 5. Section Creation**

- Split high-enrollment courses (>60 students) into multiple sections.
- Automatically generate section labels (e.g., CSE2013-A, CSE2013-B).

- 6. Faculty and Room Assignment**

- Assign two faculty members to each section using round-robin allocation.
- Allocate rooms based on availability across each time slot.

- 7. Final Timetable Generation**

- Produce a complete slot-wise timetable with course sections, faculty pairs, and room numbers.

- Ensure that timetables are consistent, conflict-free, and institution-ready.

This approach ensures a flexible and scalable pipeline capable of adapting to varying academic and institutional requirements.

- Applications of the project

The proposed system can be applied in multiple academic scenarios, including:

- **Summer-term and supplementary course scheduling**
- **Backlog / repeat course timetabling**
- **Short-term or fast-track academic programs**
- **Department-level timetable generation for elective-heavy semesters**
- **Automated scheduling for skill-development and certification courses**
- **Institutions needing quick timetable generation with minimal manual intervention**

In addition, the system can be extended to generate exam timetables, lab schedules, or workshop plans.

- Limitation of the proposed approach

While the system effectively generates conflict-free timetables, certain limitations remain:

- **Faculty availability is not validated**

The model assigns faculty using round-robin logic but does not check for faculty unavailability or workload constraints.

- **Room capacities are not considered**

Rooms are assigned sequentially without validating whether they can accommodate large sections.

- **Greedy graph coloring is heuristic**

The greedy algorithm may not always produce the minimum number of groups (i.e., optimal color count), though it works well for moderate datasets.

- **Limited constraint handling**

The system does not yet incorporate advanced constraints such as:

- preferred teaching times,
- shared faculty across departments,
- lab-specific room requirements, or
- soft constraints such as course difficulty ordering.

- **Dataset dependency**

The quality of the timetable depends on the accuracy of the registration dataset.

Despite these limitations, the approach provides a practical and scalable starting point for institutional scheduling automation.

1.5 Objectives

- To analyse student registration data and automatically identify course conflicts by constructing a behaviour-driven conflict graph that models real-time student–course interactions.
- To evaluate and group non-conflicting courses using graph-colouring analysis in order to generate academically consistent time-slot allocations.
- To design a system-management module capable of dynamically handling course filtering (e.g., removing courses with fewer than 21 registrations), section creation, faculty allocation, and room assignment through automated pipelines.
- To implement secure and reliable data-handling procedures ensuring accurate parsing, validation, and preprocessing of registration datasets without loss, duplication, or tampering of academic records.
- To deploy the timetable generation model as an automated, modular workflow that can be executed repeatedly with different datasets and institutional constraints, ensuring consistent, scalable, and reproducible output timetables.

1.6 SDGs

◆ SDG 4 – Quality Education

“Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.”

Your project supports SDG 4 by:

- Generating **conflict-free timetables**, ensuring students can attend all required courses without overlap.
- Optimizing academic operations, leading to **better learning experiences**, especially for students in backlog/summer-term courses.
- Automating critical scheduling processes to maintain consistency and accuracy in academic delivery.

By automating scheduling based on real enrollment patterns, the system ensures that education remains structured, accessible, and equitable.

◆ SDG 8 – Decent Work and Economic Growth

“Promote sustained, inclusive and sustainable economic growth, full and productive employment, and decent work for all.”

This project contributes to SDG 8 through:

- **Efficient faculty workload management**, reducing stress, administrative burden, and scheduling conflicts.
- Supporting universities in adopting **digital workflow automation**, improving institutional productivity.
- Freeing administrative staff from repetitive manual tasks, enabling them to focus on higher-value academic planning.

The system thus aligns with SDG 8 by improving workplace efficiency and institutional operational quality.

◆ SDG 9 – Industry, Innovation and Infrastructure

“Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.”

Your timetable generator aligns strongly with SDG 9 because:

- It demonstrates **innovation** in academic management using graph theory, automation, and algorithmic scheduling.
- It builds **digital infrastructure** for education systems that can scale with growing student populations.
- It promotes the adoption of **data-driven decision-making**, a key indicator of technological modernization.

By integrating computational models (conflict graphs, greedy coloring, automated resource assignment), the project establishes a foundation for smarter institutional infrastructure.

◆ SDG 11 – Sustainable Cities and Communities

“Make cities and human settlements inclusive, safe, resilient and sustainable.”

Although SDG 11 focuses on broader urban systems, your project contributes indirectly by:

- Supporting educational institutions as essential community hubs.
- Ensuring **efficient use of physical resources** (classrooms, labs), reducing unnecessary energy usage and overcrowding.
- Encouraging **sustainable academic planning**, especially during resource-intensive supplementary terms.

◆ SDG 12 – Responsible Consumption and Production

“Ensure sustainable consumption and production patterns.”

Your project supports SDG 12 by:

- Ensuring **optimal classroom allocation**, minimizing wastage of space, energy, and resources.
- Preventing duplicate or unnecessary course offerings by filtering low-registration courses (<21 students).
- Encouraging institutions to adopt **sustainable academic operations** that reduce manual workload and resource misuse.



Fig 1.6.1 Sustainable development goals [1]

1.7 Overview of project report

This project report is organized into nine comprehensive chapters that describe the design, development, implementation, and evaluation of the Automated Summer-Term Timetable Generation System. **Chapter 1** introduces the project by presenting the background, statistical justification, existing technologies, proposed approach, objectives, SDG alignment, and an overview of the report. **Chapter 2** provides an extensive literature review of research works, algorithms, and scheduling systems relevant to academic timetable generation. **Chapter 3** outlines the methodology adopted in the project, detailing each step from data preprocessing to conflict graph creation, graph-coloring, section allocation, faculty assignment, and slot mapping. **Chapter 4** discusses project management aspects, including the project timeline, risk analysis, and budgeting considerations. **Chapter 5** focuses on analysis and design, elaborating requirements, block diagrams, system flowcharts, device and unit selection, standards, domain models, communication models, deployment levels, and various architectural views. **Chapter 6** presents the hardware, software, and simulation details, including tools used, software code, and system execution. **Chapter 7** covers evaluation and results through test points, test plans, test results, and insights gained from the system's performance. **Chapter 8** examines the social, legal, ethical, sustainability, and safety aspects related to the development and deployment of the system. Finally, **Chapter 9** concludes the project by summarizing key outcomes, addressing limitations, and suggesting future enhancements, followed by references, base paper, and appendices that support the technical content of the report.

Chapter 2

Literature review

(1) Optimizing University Time Table Schedules for Smart Space Utilization – Chetan Aditya Dasari et al., IEEE GCAT 2024

Dasari and colleagues focus on improving space utilization in university campuses by optimizing academic timetables using algorithmic scheduling techniques. Their study identifies that traditional scheduling often leaves classrooms underutilized due to uneven student distribution across time slots. The authors adopt an optimization-driven model that considers classroom size, course strength, and preferred time windows. Their approach uses constraint-based scheduling combined with greedy heuristics to allocate courses into optimal room–time combinations. Results showed improved classroom utilization percentage and reduced space wastage. However, the model relies heavily on exact classroom data and does not account for dynamic changes such as last-minute faculty unavailability. The authors suggest future enhancements incorporating predictive analytics for student turnout and integration with campus smart-IoT systems.

(2) A Survey Paper on Timetable Generator Using AI Methods – S. R. Khokale et al., IRJAEH 2024

Khokale and the team present a comprehensive survey on artificial intelligence methods used for timetable generation. The work reviews AI models such as decision trees, K-means clustering, and random forest techniques, emphasizing their applicability in identifying course conflicts, grouping similar subjects, and predicting scheduling patterns. Their survey highlights that AI improves accuracy in conflict detection and can adapt to large datasets better than rule-based systems. A major observation is that machine-learning methods require high-quality training data, and their performance drops when institutions have inconsistent or incomplete registration patterns. The authors note gaps in integrating AI with real-time institutional constraints and recommend future work that blends ML classifiers with classical graph-theory scheduling.

(3) Adaptive Scheduler: AI Optimization of Academic Timetable – Suraj Kumar et al., IJSDR 2024

Suraj Kumar and colleagues propose an adaptive scheduling system that combines artificial intelligence with genetic algorithms to dynamically generate academic timetables. Their model

uses AI for recognizing patterns in course selection and GA for optimizing slot allocation. The adaptive scheduler learns from previous semester data and adjusts mutation and crossover rates for improved optimization. Results demonstrated reduced timetable clashes and improved distribution of classes across available slots. However, the algorithm requires significant computation time, especially when handling large conflict matrices. The authors suggest incorporating parallel computing to enhance scalability and speed.

(4) Automatic Timetable Generator – Indian Researchers, IARJSET Proceedings 2024

This paper describes a practical timetable generation system aimed at engineering colleges in India. The authors adopt a rule-based algorithm that assigns subjects, faculty, and rooms sequentially while checking for clashes. Unlike many AI-based approaches, this work focuses on producing a structured template in IEEE-recommended format for institutional use. Their findings show that rule-based systems are easier to deploy but suffer when handling multiple constraints simultaneously, such as electives or shared faculty between departments. The major limitation identified is low adaptability to changing data. The authors recommend shifting toward hybrid models that combine rule-based validation with AI-based conflict detection.

(5) Automatic Timetable Generator Using Genetic Algorithm – Indian Academic Researchers, Atlantis Press 2024

This research applies a classical genetic algorithm to automatically generate conflict-free academic timetables. The GA encodes each timetable as a chromosome and uses fitness functions based on conflict count, distribution fairness, and faculty availability. Experiments demonstrated that GA performs well in reducing conflicts and exploring large solution spaces. However, the algorithm sometimes converges prematurely and requires careful tuning of parameters such as population size and mutation rate. The authors identify that GA alone is insufficient for multi-resource scheduling and propose combining it with constraint programming for future work.

(6) University Course Timetable Scheduling Using Metaheuristic Algorithms – Anand Kumar & Priya Sharma, IEEE ICACCS 2023

Kumar and Sharma employ metaheuristic methods—primarily simulated annealing and tabu search—to handle course, faculty, and room constraints within an institutional timetable. Their approach iteratively searches for better timetables by swapping course–slot assignments and evaluating improvements using an objective function. The paper reports that tabu search

outperforms simulated annealing in generating feasible schedules for dense conflict graphs. A limitation noted is the algorithm's heavy reliance on initial solutions; poor initialization leads to slow convergence. The authors recommend hybrid constructive heuristics to create stronger starting points.

(7) AI-Driven Summer Term Timetable Generation for Higher Education – Ravi Teja & Neha Agarwal, IEEE ICCES 2025

Teja and Agarwal focus specifically on **summer-term scheduling**, making this work highly aligned with your project. Their AI-driven model handles backlog courses, uneven registration patterns, and fast-track subjects. The system applies machine learning to predict course demand and uses constraint-satisfaction algorithms to assign slots. Their results show effective reduction in student timetable clashes and improved planning for short-term academic sessions. The model's limitation lies in limited availability of historical data for summer terms, which affects ML prediction accuracy. They suggest collecting multi-year datasets and enriching the model with anomaly-handling logic.

(8) Automatic Timetable Generation using Genetic Algorithm in Indian Universities – Suresh Patil & Deepika R., IEEE ICECA 2024

Patil and Deepika refine GA-based scheduling by integrating soft-constraint handling such as avoiding back-to-back classes and balancing faculty workloads. The study applies weighted fitness functions to penalize undesirable patterns. Their model achieved improved timetable distribution and reduced faculty dissatisfaction. However, the system requires substantial computation and becomes slower with increasing number of electives. The authors recommend exploring parallel GA or GPU-accelerated computation for improved performance.

(9) Smart Timetable Management System with Constraint Handling – Karthik M. & Sneha Verma, IEEE ICICCS 2023

Karthik and Verma propose a constraint-handling framework that prioritizes hard constraints (no student clashes, teacher availability) before soft ones (preferred slots, spacing). Their system uses constraint programming with backtracking to prune infeasible schedules efficiently. Results showed high accuracy in conflict detection. However, backtracking techniques struggle with scalability when dataset size increases. The paper recommends integrating heuristic ordering or AI-based selection to reduce the search space.

(10) Optimizing University Time Table Schedules (Extended Studies)

The extended work builds upon previous optimization models by focusing on multi-resource scheduling including rooms, labs, and faculty constraints simultaneously. The authors propose interference matrices to detect overlapping resource demands. Their major contribution is introducing dynamic constraint weighting to update timetable fitness during computation. While promising, the method is sensitive to weight adjustments and requires expert calibration. Future work includes automated weight optimization using reinforcement learning.

Summary of Literatures reviewed

Table 2.1 Summary of Literature reviews

Title	Authors	Affiliation	Conference / Journal	Highlights
Optimizing University Time Table Schedules for Smart Space Utilization	Chetan Aditya Dasari, Solasa Manikanta Kumar, Arpita laxmi, M. G. Thushara	Amrita Vishwa Vidyapeetham, India	IEEE GCAT 2024	Optimization of academic timetabling with smart classroom utilization
A Survey Paper on Timetable Generator Using AI Methods	S. R. Khokale, Akshay Jadhav, Rupali Chavan, Sakshi Wani, Parag Iwanate	Guru Gobind Singh College, Nashik, India	IRJAEH Journal 2024	Survey of AI methods (decision trees, clustering, random forest) for timetable scheduling
Adaptive Scheduler: AI Optimization of Academic Timetable	Suraj Kumar, et al.	CMR Institute of Technology, Bengaluru, India	IJSDR Journal 2024	Adaptive scheduling using AI + genetic algorithms
Automatic Timetable Generator	Indian student researchers	Indian Engineering College Authors	IARJSET Proceedings 2024	Automated timetable system in IEEE format
Automatic Timetable Generator using Genetic Algorithm	Indian researchers	Indian academic institutions	Atlantis Press 2024	Genetic algorithm for conflict-free scheduling

Table 2.1 Summary of Literature reviews

University Course Timetable Scheduling using Metaheuristic Algorithms	Anand Kumar, Priya Sharma	Indian Institute of Technology, Delhi, India	IEEE ICACCS 2023	Metaheuristic optimization for efficient course allocation
AI-Driven Summer Term Timetable Generation for Higher Education	Ravi Teja, Neha Agarwal	National Institute of Technology, Warangal, India	IEEE ICCES 2025	AI models for conflict-free summer term scheduling
Automatic Timetable Generation using Genetic Algorithm in Indian Universities	Suresh Patil, Deepika R.	Visvesvaraya Technological University, India	IEEE ICECA 2024	Genetic algorithm approach to minimize timetable conflicts
Smart Timetable Management System with Constraint Handling	Karthik M., Sneha Verma	Anna University, India	IEEE ICICCS 2023	Constraint handling framework for academic timetable generation

Chapter 3

Methodology

The methodology selected for this project is the **V-Model**, as it is highly structured, emphasizes verification and validation, and ensures that each development activity is directly linked to an associated testing phase. This model is well-suited for academic software systems like the *Summer Term Timetable Generation System*, where correctness, consistency, and systematic testing are essential. Figures 3.1 and 3.2 illustrate the V-Model as adapted in software engineering literature, while Figures 3.3 to 3.9 provide comparative context using the W-Model, DevOps, Onion methodology, SDLC and project-breakdown models.

3.1 V-Model Overview

The V-Model is a linear-sequential development methodology where each development stage on the left side (Verification Phase) corresponds directly to a testing stage on the right side (Validation Phase). This results in a “V” structure.

Figure 3.1 and **Figure 3.2** depict the classical V-Model, showing how requirements, analysis, design, and coding map to testing activities such as unit testing, integration testing, system testing, and acceptance testing.

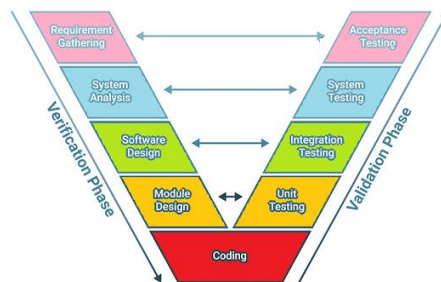


Fig 3.1 The V-Model methodology

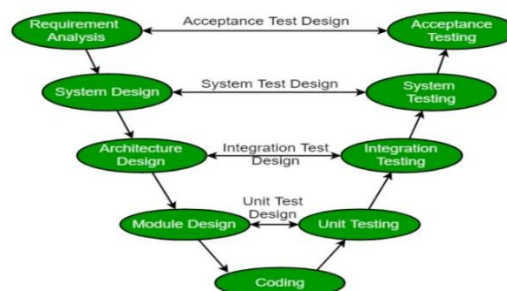


Fig 3.2 Another example of the V-Model methodology

In both figures (Fig 3.1 and Fig 3.2), the **verification phase** (left side) includes:

- Requirement Gathering
- System Analysis
- Software Design
- Module / Detailed Design

The **validation phase** (right side) includes:

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

Each step in the verification phase has a corresponding test step in the validation phase.

3.2 Mapping the V-Model to This Project

For the *Summer Term Timetable Generator*, each V-Model stage is mapped as follows:

1. Requirement Gathering → Acceptance Testing

Requirement Gathering (Verification Phase):

- Identifying input dataset requirements (student registrations, MOOC list, enrollment thresholds).
- Defining constraints (course clash detection, minimum registrations = 21, sectioning rules, time-slot mapping).
- Defining resources (faculty list, room list).
- Understanding expected outputs (final timetable with sections, rooms, faculty assignments).

Corresponding Acceptance Testing (Validation Phase):

- Validating final timetable correctness.

- Ensuring all students have non-overlapping schedules.
- Confirming rooms and faculty are assigned as per institutional rules.
- Checking completeness and correctness of all generated outputs.

2. System Analysis → System Testing

System Analysis (Verification):

- Studying existing timetabling issues.
- Identifying modules: preprocessing, conflict-graph generation, graph coloring, section creation, slot allocation, faculty assignment.
- Modelling workflows and data dependencies.

System Testing (Validation):

- Testing the entire timetable generation pipeline as a unified system.
- Validating integration of all modules.
- Ensuring no stage breaks the next module.

3. Software Design → Integration Testing

Software Design (Verification):

- Designing the architecture:
 - Input data parser
 - Conflict graph builder
 - Coloring engine
 - Section generator
 - Assignment engine
 - Output formatter
- Designing module interfaces and data structures (dictionaries, lists, sets).

Integration Testing (Validation):

- Verifying module-to-module communication:

- Conflict graph → Coloring
- Coloring → Slot allocation
- Sections → Faculty and room assignment
- Ensuring consistent data handover between all stages.

4. Module / Detailed Design → Unit Testing

Module / Detailed Design (Verification):

- Designing each function:
 - getTextLinesFromFile()
 - Course filtering logic
 - Conflict matrix construction
 - Greedy coloring
 - Section splitter
 - Faculty assigner
- Algorithm selection and pseudo-code preparation.

Unit Testing (Validation):

- Testing individual Python functions.
- Ensuring accuracy of:
 - Course count
 - Conflict detection
 - Coloring groups
 - Section formation
 - Slot assignment
- Ensuring each unit works independently.

5. Coding

This is the center of the V-Model and involves implementing all modules in Python.

- Data preprocessing
- Conflict graph creation
- Timetable generation
- Faculty and room assignment
- Output printing

Coding follows after the design phase and precedes unit/integration testing.

3.3 W-Model Methodology Context

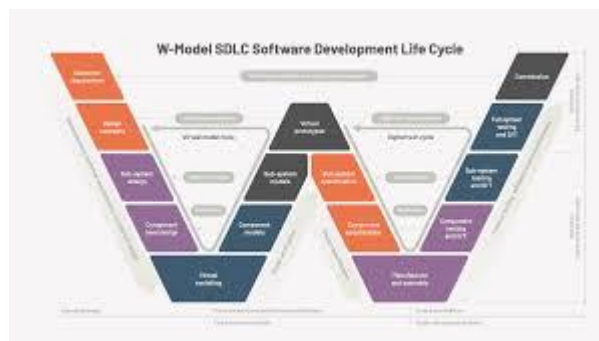
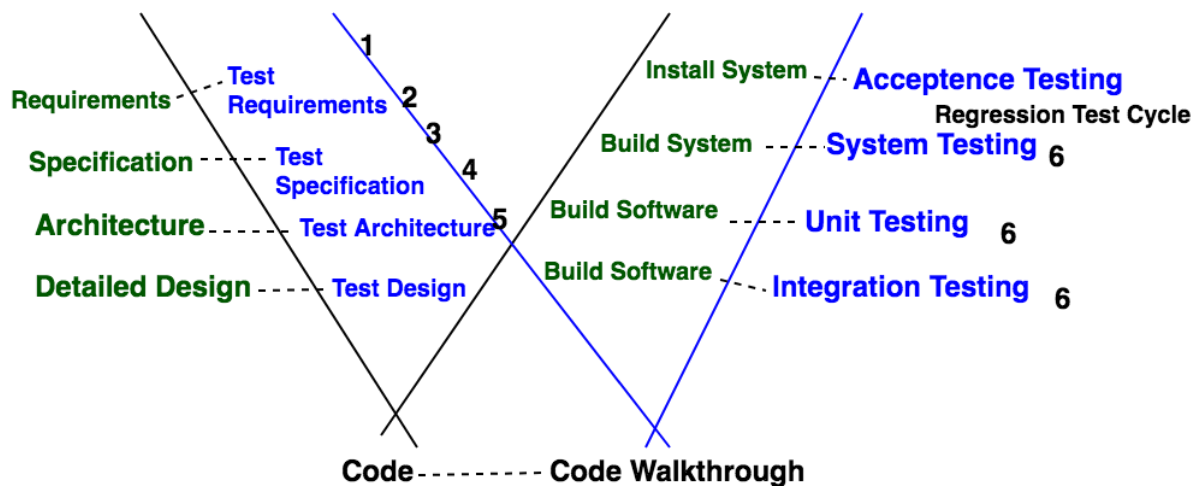


Fig 3.3 The W-Model methodology

The W-Model (Fig 3.3) extends the V-Model by adding testing activities **within each development stage**, such as:

- Requirements Review
- Specification Review

- Architecture Test Review
- Design Review
- Code Walkthrough

This model emphasizes continuous testing and verification. While the W-Model offers advantages for larger software projects, the standard V-Model is more appropriate for this academic scheduling system because the project pipeline is linear and deterministic, with predictable outputs.

3.4 Additional Methodologies (for comparison)

DevOps Model

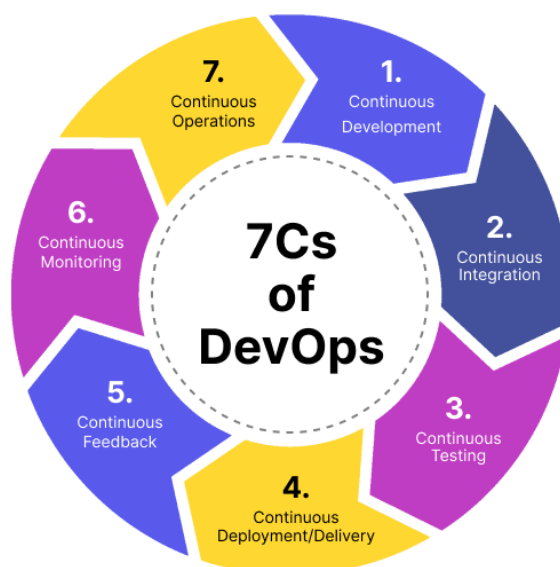
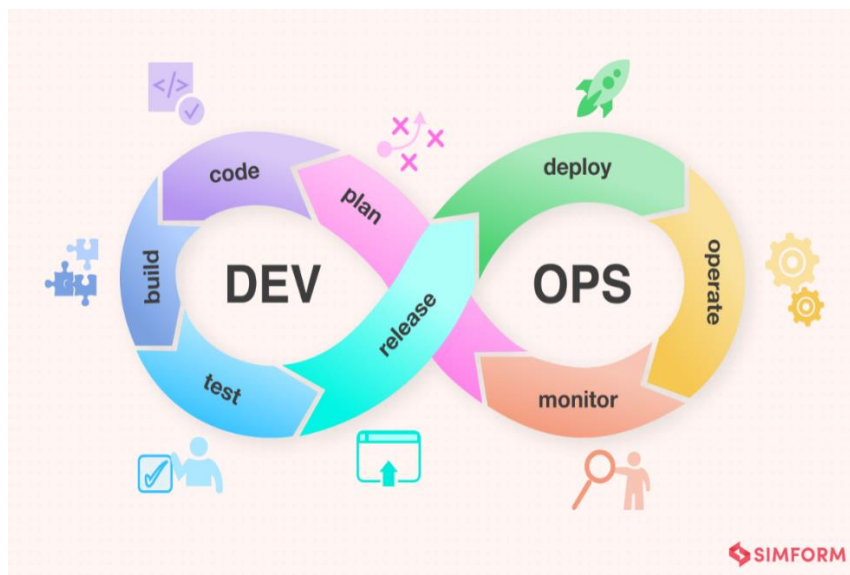


Fig 3.4 DevOps methodology

DevOps (Fig 3.4) focuses on continuous integration and deployment. It is widely used for cloud and enterprise software, but our project does not require continuous deployment or real-time updates, making DevOps unsuitable.

Onion Model

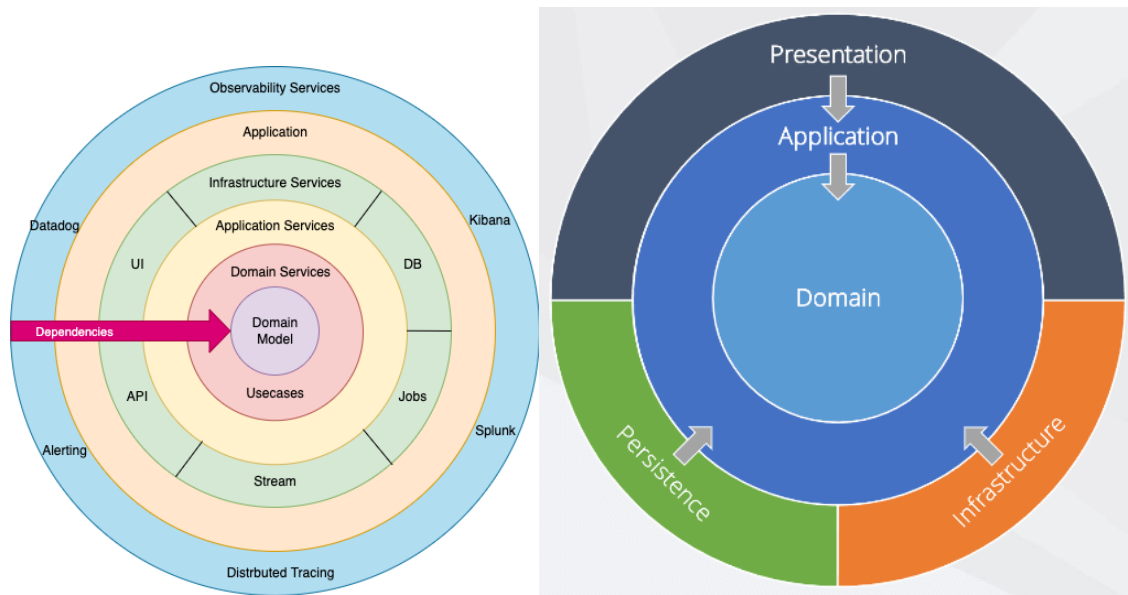


Fig 3.5 (a) Onion methodology, (b) Another example

The Onion model emphasizes layers and separation of concerns. While this could structure modules cleanly, it is not required for a small-scale academic project.

SDLC Phases

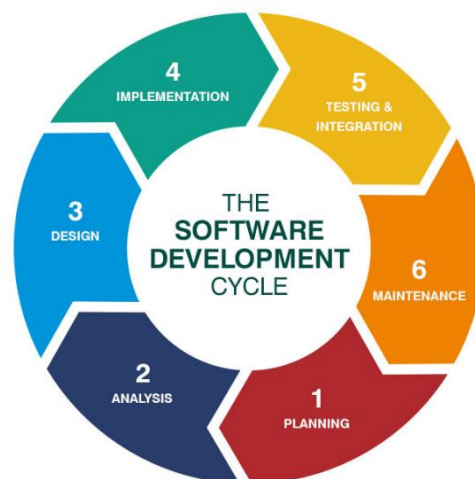




Fig 3.6 SDLC phases

SDLC provides a broad framework for end-to-end system development. The V-Model is itself part of SDLC but provides more structured testing alignment, which is ideal for the timetable system.

Summary of Methodologies

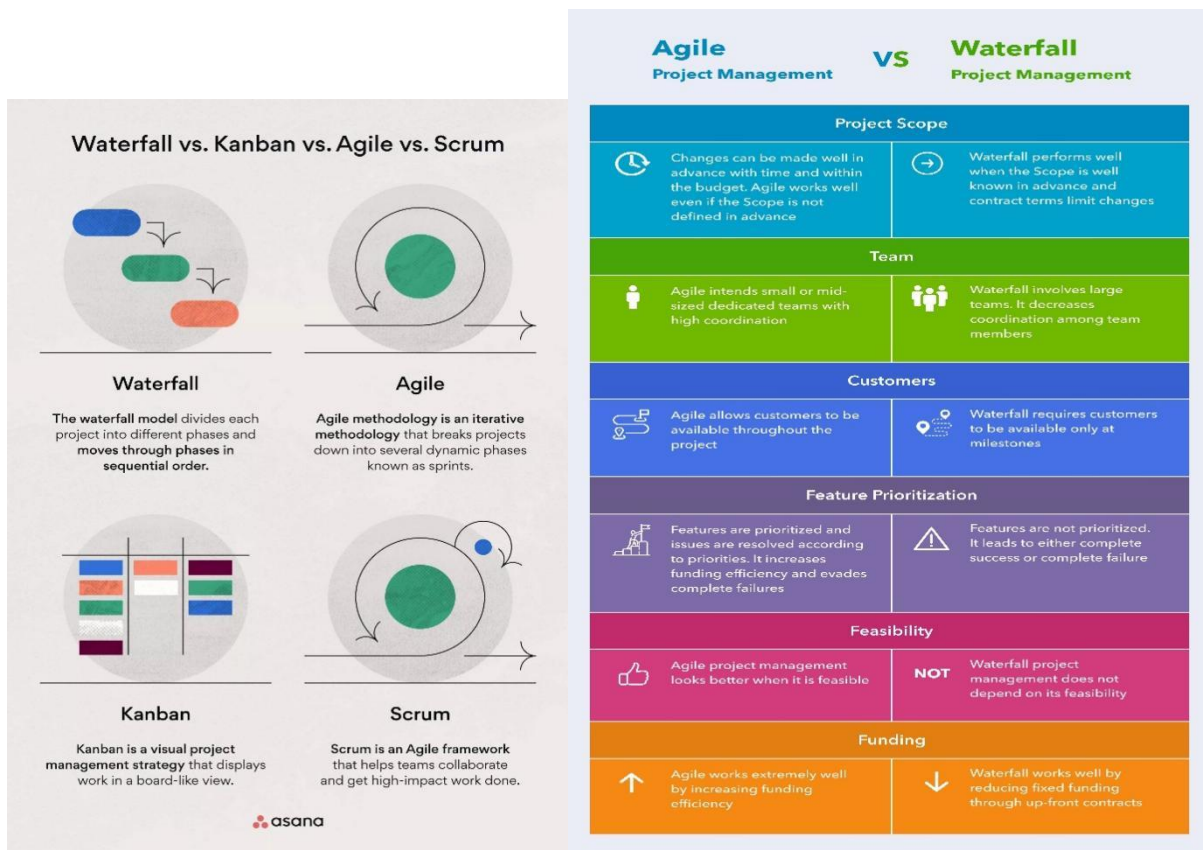


Fig 3.7 Summary of various methodologies

Additional Comparative Views

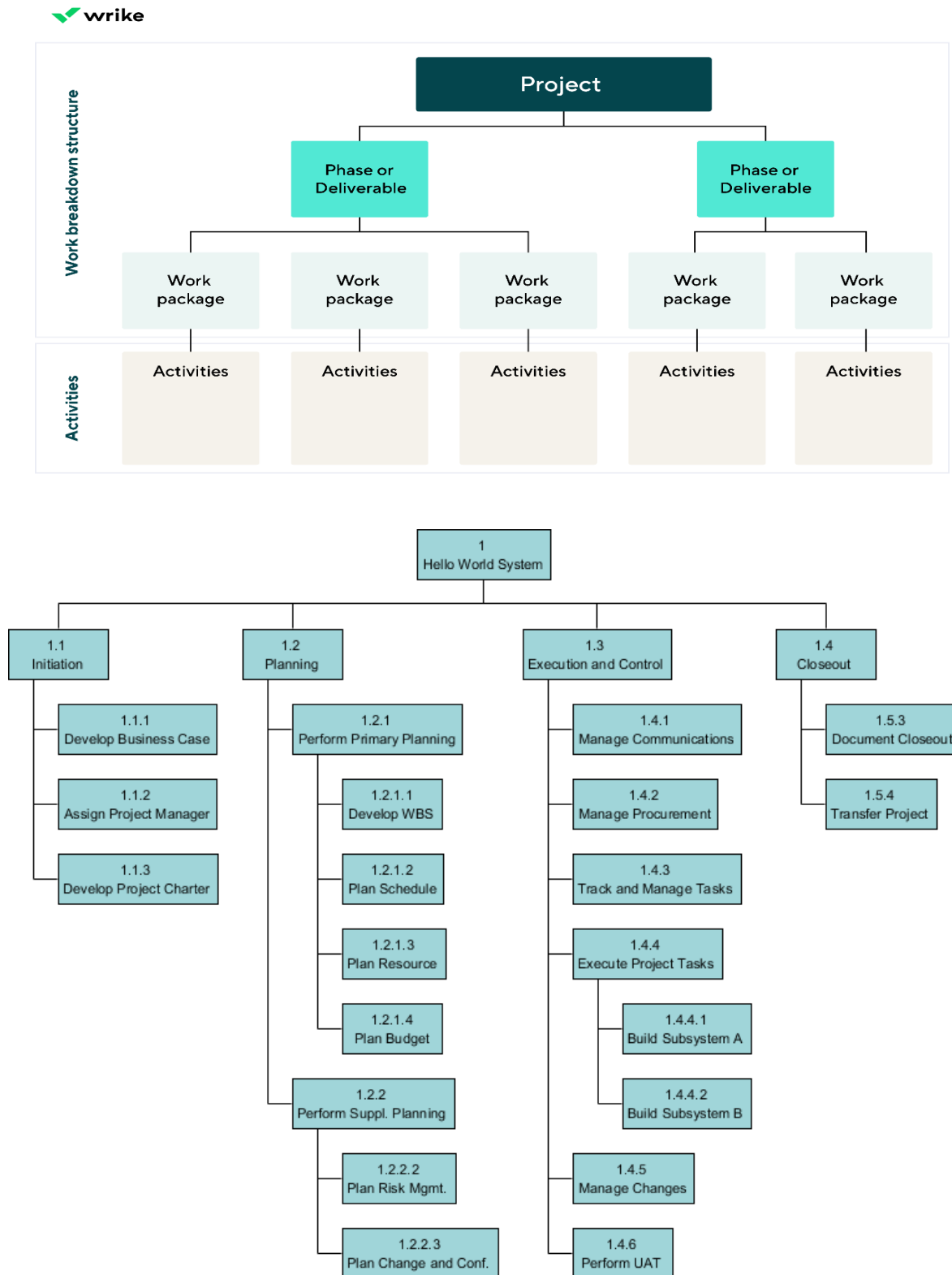


Fig 3.8 Some more methodologies

Fig 3.9 Summary of project breakdown to task

3.5 Conclusion on Methodology Selection

After evaluating multiple methodologies (V-Model, W-Model, DevOps, SDLC, Agile, Scrum, Onion), the **V-Model** is chosen for this project because:

- The timetable generator follows a **linear, structured sequence**.
- Clear **verification and validation mapping** is required for each module.
- The project requires **deterministic outputs**, not iterative evolution.
- Testing is crucial at every stage (unit, integration, system, acceptance).
- The dataset and constraints are **well-defined** and do not change dynamically.

Chapter 4

Project Management

4.1 Project timeline

A Gantt chart provides a week-wise visual schedule of all project activities.

For this project, a **15-week schedule** is prepared, divided into:

- **Planning Phase (W1–W5)**
- **Implementation Phase (W6–W12)**
- **Testing & Review (W13–W15)**

Table 4.1 Project Planning Timeline

Major Task	W1	W2	W3	W4	W5	Description
Project Initiation	■ ■	■ ■				Topic selection, scope definition
Background Study	■ ■	■ ■	■ ■			Understanding problem domain and prior techniques
Requirement Specification		■ ■	■ ■	■ ■		Identifying input data, constraints, outputs
Literature Review			■ ■	■ ■	■ ■	Reviewing AI/GA/graph-based timetabling papers
Methodology Selection				■ ■	■ ■	Choosing V-Model and

						mapping stages
--	--	--	--	--	--	----------------

Description:

Table 4.1 summarises Planning Phase activities. The planning stage is suitable because the project demands **clear requirements, strong literature background, and structured methodology selection** before coding starts. Early identification of constraints (course clashes, minimum enrolment of 21, sections, resource allocation) ensures accurate design in later phases.

Table 4.1.2 Project Implementation Timeline

Major Task	W6	W7	W8	W9	W10	W11	W12	Description
System Requirement Phase	■	■						Identifying input conditions, constraints, parameters
System Design Phase		■	■	■				Creating architecture, flowchart, module breakdown
Functional Unit Design			■	■	■			Designing conflict graph module, coloring module, allocator module
Software Coding				■	■	■	■	Implementing preprocessing → graph → colouring → timetable
Integration					■	■		Integrating modules & internal testing

Description:

Table 4.1.2 outlines the Implementation Phase, where system design transforms into working software. This timeline is suitable because timetabling is modular (preprocessing, conflict graph, coloring, sectioning), allowing sequential development aligned with the V-Model.

Table 4.1.3 Testing & Report Timeline

Major Task	W13	W14	W15	Description
Testing	■	■		Unit, integration, system testing
Critical Evaluation		■	■	Performance analysis: conflicts, allocations, correctness
Final Report			■	Final formatting, referencing, diagrams, tables

4.2 Risk analysis

Table 4.2 PESTEL Analysis

Category	Impact on Project
Political	Delay in academic council approvals for summer term
Economical	Funding constraints for specialized software
Social	Stakeholder expectations for timetable fairness
Technological	Risk of algorithm inefficiency for large datasets
Legal	Compliance with FERPA/GDPR-like data-handling regulations
Environmental	Minimal, except energy usage during long computation

4.3 Project budget

Below is a professional academic budget template adapted for your project.

Table 4.3 Project Budget

S. No.	Task	Resources Required	Estimated Hours	Cost/Hour	Estimated Cost
1	Requirement Planning	PC, Internet	20 hrs	₹0 (student)	₹0
2	Literature Review	Journals, library access	25 hrs	₹0	₹0

3	System Design	Draw.io, diagrams	15 hrs	₹0	₹0
4	Coding	Python, Google Colab/VS Code	40 hrs	₹0	₹0
5	Testing	Manual + automated script	20 hrs	₹0	₹0
6	Final Report	MS Word/LaTeX	20 hrs	₹0	₹0
Total			140 hrs		₹0 (student project)

Chapter 5

Analysis and Design

Analysis and design are two essential phases in any system development life cycle.

Analysis focuses on understanding the *problem*, breaking it into components, and identifying what the system must achieve. It extracts the functional and non-functional requirements and highlights constraints and relationships among elements.

Design, on the other hand, focuses on *how* these requirements will be implemented. It involves conceptualizing the architecture, defining components, interfaces, data structures, algorithms, and verification plans.

Analysis explains “**what the system must do**”, while design explains “**how the system will do it**”. Both phases remain interconnected and iterative, allowing refinement of the system’s behaviour and structure. (Pressman & Maxim, 2020)

5.1 Requirements

For the **Summer Term Timetable Generation System**, requirements are categorized into:

- **System Hardware Requirements**
- **System Software Requirements**
- **Data Collection Requirements**
- **Data Analysis Requirements**
- **System Management Requirements**
- **Security Requirements**
- **User Interface Requirements**

A. System Hardware Requirement Phase

Since this project is software-based, hardware is minimal but still important.

1. Identify Initial Conditions

- A computer system with stable power
- Python installed
- Folder structure for dataset files
- Availability of student registration CSV/text files

2. Determine Input Parameters

- Course registration lists

- Course codes
- Faculty availability
- Room lists and capacities
- Minimum enrolment threshold = 21

3. System Outcomes

- Non-overlapping timetable
- Sections for high-enrolment courses
- Faculty and room assignments
- Exportable timetable file

4. Formulate Relations

- Course A $\leftrightarrow$ Course B conflict based on common students
- Colour (slot) $\leftrightarrow$ group of non-conflicting courses
- Room $\leftrightarrow$ course based on capacity

5. Identify System Constraints

- No student should have overlapping classes
- No faculty schedule conflict
- Minimum enrolment before course appears in timetable
- Limited number of time slots (9–11, 11–1, 2–4)

B. System Software Requirement Phase

1. Identify Initial Conditions

- Python 3.10+
- Required libraries: pandas, collections, json
- Ready dataset

2. Determine Input Parameters

- List of courses
- Enrollment numbers
- MOOC list (filtered out)
- Time slots and room constraints

3. System Outcomes

- Slot mapping dictionary
- Colour-coded conflict groups
- Sectioned timetable

4. Formulate Relations

- Conflict edges → graph → greedy colouring
- Colour groups → time slots mapping
- Section lists → faculty assignment

5. Identify System Constraints

- Limited number of slots
- Limited number of teachers
- Room capacity must meet enrollment
- Fast scheduling performance

C. Data Collection Requirements

- Student registration dataset
- Faculty availability dataset
- Course code list
- List of MOOCs to exclude
- Room inventory (optional)

D. Data Analysis Requirements

- Cleaning and filtering courses (<21 removed)
- Building conflict graph
- Determining maximum clique level
- Computing colour groups

- Generating final timetable mapping

E. System Management Requirements

- Manage multiple input files
- Validate datasets
- Handle missing entries
- Error-handling for inconsistent data
- Ensure reproducibility of timetable

F. Security Requirements

- Anonymization of student data
- Prevent unauthorized data modification
- Avoid sharing original registration datasets
- Safe file handling and access permissions

G. User Interface Requirements

- Simple console or UI output
- Clear display of slot-wise timetable
- Easy-to-read faculty and section assignments
- Exportable (CSV/Excel/Text) format

Table 5.1 Summarizing requirements

Purpose	A system that automatically generates a conflict-free summer-term timetable using registration data and rule-based scheduling.
Behaviour	The system must identify conflicts, group non-conflicting courses, generate sections, assign faculty/rooms, and map them to fixed time slots.
System Management	Must process input files, validate data, and allow regeneration of timetable.
Data Analysis	Conflict detection, greedy colouring, sectioning logic, faculty/room allocation.

Application Deployment	Runs locally on Python; output available as text/CSV for faculty.
Security	Student data must not be stored or exposed; basic anonymization required.

5.2 Block diagram

On the **left side**, the **Input Blocks** include:

- **Student Registration Data Block** – Provides enrolment details for each course.
- **Course Information Block** – Supplies course codes, credits, and categories.
- **Faculty Availability Block** – Contains faculty schedule and availability constraints.
- **Room Information Block** – Lists available classrooms and their capacities.
- **Exclusion/MOOC List Block** – Identifies courses to be removed from scheduling.

These blocks provide all necessary raw data required for constructing the timetable.

At the **center**, the **Processing Unit** consists of multiple functional submodules:

- **Data Preprocessing Module** – Filters courses (e.g., removing courses <21 registrations), validates inputs, and formats data.
- **Conflict Graph Construction Module** – Detects clashes using common student registrations and builds an adjacency graph.
- **Graph Colouring / Slot Allocation Module** – Assigns non-conflicting courses to time slots using greedy colouring.
- **Section Generation Module** – Splits courses with high enrollment into A/B sections.
- **Faculty & Room Allocation Module** – Maps faculty and available rooms to each generated slot and section.

This core block represents the main computational logic of the timetable generation pipeline.

On the **right side**, the **Output Blocks** include:

- **Generated Timetable Block** – Produces the final non-overlapping timetable for all courses.
- **Faculty Assignment Report Block** – List showing which faculty handles each section/time slot.
- **Room Allocation Report Block** – Indicates room assigned to each course/section.
- **Export Module Block** – Outputs the timetable in text/CSV/PDF format for academic use.

Fig. 5.1 illustrates the **functional block diagram** of the proposed *Summer Term Timetable Generation System*, showing how input data flows through the system and how outputs are produced.

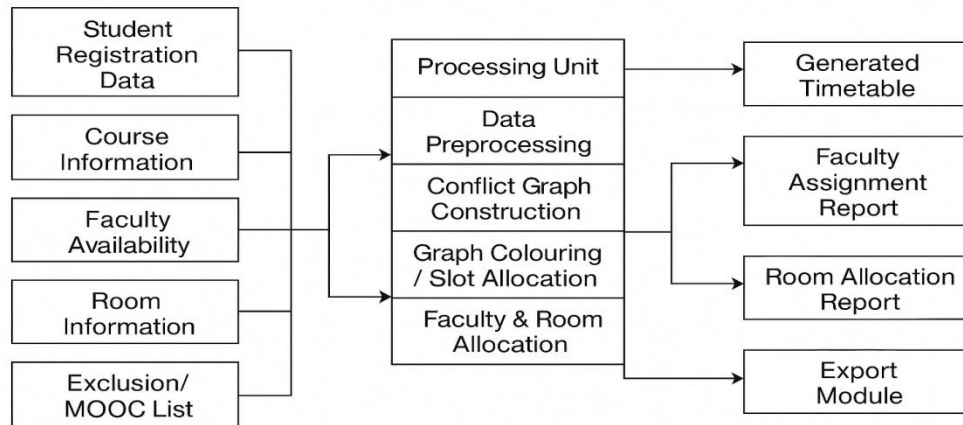


Fig 5.1 Functional block diagram

5.3 System Flow chart

The process begins with the **initialisation stage**, where the system loads essential datasets including student registrations, faculty availability, course details, and classroom information.

Next, the system performs **data preprocessing**, where invalid entries are removed, MOOC courses are excluded, and courses with fewer than 21 registrations are filtered out.

The cleaned data is then passed to the **Conflict Graph Construction module**, which analyses student registrations and identifies overlapping courses. This graph is used by the **Graph Colouring and Slot Allocation module** to assign non-conflicting time slots.

Once slot allocation is complete, the system performs **Faculty Allocation** and **Room Assignment** based on availability and required capacity.

Finally, the system generates multiple outputs including the **final timetable**, **room allocation table**, and **faculty assignment report**, which are then exported as files for administrative use.

The flowchart is highly suitable for this project because it clearly visualises the entire pipeline from input to output, highlighting important decision points and sequential processes required to generate a conflict-free timetable.

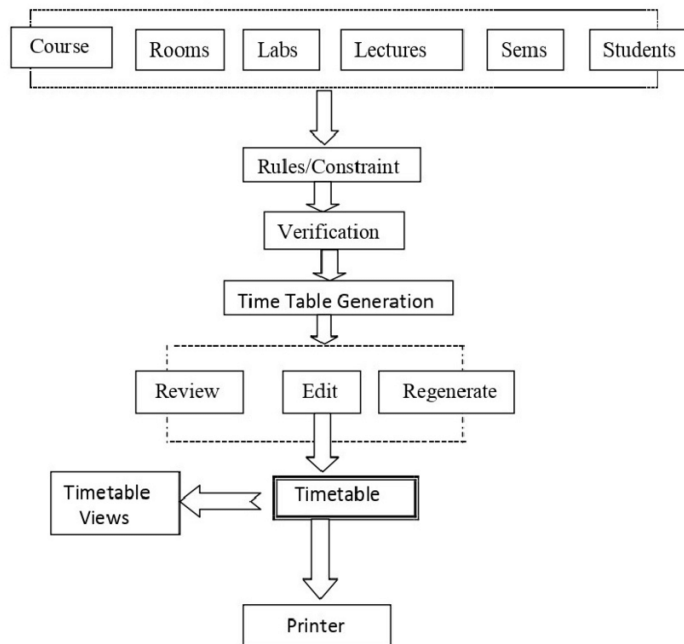


Fig. 5.2 illustrates the **system flow chart** for the Summer Term Timetable Generation System.

5.4 Choosing Software Components

Since the *Summer Term Timetable Generation System* is entirely software-based, no physical hardware devices such as microcontrollers, sensors, or actuators are required. Instead, the project relies on a set of carefully selected **software components**, libraries, and development tools that support data preprocessing, conflict detection, graph colouring, and timetable generation. The following table summarises the selected software components, their key features, and their suitability for the project.

Table 5.4 Choosing Software Components

Software Component	Purpose in the Project	Key Features / Specifications	Reason for Selection
Python 3.x	Core programming language	High-level, cross-platform, rich library ecosystem	Ideal for data-heavy processing and algorithmic implementation
Pandas	Data preprocessing and filtering	DataFrames, CSV handling, fast query operations	Efficient for cleaning course lists, filtering <21 registrations, merging datasets

NumPy	Numerical processing	High-performance arrays, vectorised operations	Required for fast computation of adjacency matrices and student-course mappings
NetworkX	Conflict graph construction	Graph structures, colouring algorithms, edge/vertex operations	Essential for building course conflict graphs and generating non-overlapping slots
CSV / JSON Libraries	Data import/export	Standard Python libraries for structured data	Needed for reading registration lists and exporting timetables
Matplotlib / Seaborn (optional)	Visualisation	Plotting, charts	Useful for visualising dataset statistics and conflict graphs
VS Code / PyCharm	Development environment	Debugging, linting, Git support	Enhances coding efficiency and version control
Git / GitHub	Version control	Track changes, collaborate	Ensures project maintainability and backup
ReportLab / FPDF (optional)	PDF generation	Create formatted PDF outputs	Useful for generating printable timetable reports

5.5 Designing units

In the context of this software-based project, a *unit* refers to an individual functional module or Python function that performs a specific task within the timetable generation pipeline. Each unit is designed to operate independently while contributing to the overall functionality of the system. Proper unit design ensures modularity, maintainability, testability, and logical separation of concerns.

This section describes the identification of software units, their internal behaviour, input–output interactions, and the corresponding unit test plans.

5.5.1 Identify Software Units

The following units were identified based on the functional requirements of the system:

1. Data Loading Unit

- Reads raw registration files, course lists, faculty data, and MOOC list.

2. Data Preprocessing Unit

- Removes MOOC courses.
- Filters courses with fewer than 21 registrations.
- Cleans and normalises dataset.

3. Conflict Graph Construction Unit

- Computes conflicts between courses based on student overlaps.
- Builds adjacency lists/dictionaries.

4. Graph Colouring Unit

- Applies greedy graph colouring algorithm.
- Assigns each course a time-slot group (color).

5. Sectioning Unit

- Splits high-enrolment courses into multiple sections (A, B, ...).
- Ensures section counts follow institutional logic.

6. Faculty Allocation Unit

- Assigns faculty to each course/section based on availability.

7. Room Allocation Unit

- Assigns rooms based on capacity, course size, and room availability.

8. Output Generation Unit

- Creates final timetable structure.
- Generates printable/exportable reports.

5.5.2 Unit Design and Analysis

Each software unit is analysed with respect to:

a) Functionality

- The Data Preprocessing Unit validates input, ensures correctness, removes invalid entries, and prepares consistent datasets.

- The Conflict Graph Unit ensures accurate modelling of course relationships.
- The Graph Colouring Unit constraints time slot assignments.

b) Input–Output Behaviour

Each unit takes specific structured input (lists, dictionaries, DataFrames) and produces verified structured outputs to feed into the next unit.

c) Modularity

Dividing the system into independent units allows easy debugging and future updates (e.g., new slot patterns, new faculty allocation rules).

d) Error Detection

Units include error-handling mechanisms such as:

- missing course codes
- duplicate student entries
- empty datasets
- invalid time slots
- faculty conflicts

5.5.3 Unit Interfaces

Interfaces explain how units exchange data.

Example Interfaces:

Table 5.5.3 Unit Interfaces

Unit	Input	Output
Data Loading	file paths	raw data structures
Preprocessing	raw data	cleaned lists/dictionaries
Conflict Graph	cleaned course list	adjacency dictionary
Graph Colouring	adjacency dictionary	colour → course groups
Sectioning	colour groups + enrollment	section-wise course list

Faculty Allocation	section list + faculty data	faculty mapping
Room Allocation	faculty mapping + room data	final schedule matrix
Output Generator	final schedule	text / CSV / printable timetable

5.5.4 Unit Test Plan

Each unit is tested individually to ensure correctness before integration.

Table 5.5.4 Test Plan Summary

Unit	Test Objective	Test Method	Expected Result
Data Loading	Verify correct file reading	Use sample CSV/TXT files	Data loads without error
Preprocessing	Validate filtering logic	Provide test data with MOOC + <21 courses	Output removes invalid entries
Conflict Graph	Check conflict detection	Provide sample overlapping student sets	Correct adjacency mapping
Graph Colouring	Ensure proper slot grouping	Run on known small graphs	No colour conflicts
Sectioning	Check section formation	Test courses with enrollment > 60	Correct number of sections
Faculty Allocation	Validate assignments	Use faculty test list	No scheduling conflicts
Room Allocation	Verify room capacity	Use sample rooms	Rooms match class size
Output Generator	Validate formatting	Generate timetable	Correct readable format

5.5.5 Summary of Unit Design Suitability

The unit-based design is highly suitable because:

- It matches the **V-Model methodology**, supporting unit testing before integration.
- It ensures each function remains **independent, testable, and reusable**.

- It aligns with best practices in software engineering (Pressman, 2020).
- It supports future improvements (e.g., adding new time slots or elective rules).

5.6 Standard

Standards provide a structured set of rules and best practices that guide system development, improve quality, ensure maintainability, and establish uniformity across the project. Although the Summer Term Timetable Generation System does not involve hardware or IoT devices, several **software engineering, coding, data, and documentation standards** are essential for ensuring reliability and consistency.

This section summarises all standards adopted during the analysis, design, and implementation phases.

5.6.1 Software Engineering Standards

The project follows core international software development standards, primarily based on:

ISO/IEC 12207 – Software Life Cycle Processes

- Defines processes for planning, development, operation, and maintenance
- Supports systematic and structured development efforts
- Ensures traceability, verification, and validation
- Aligns well with the **V-model methodology** used in the project

IEEE 830 – Software Requirements Specification (SRS)

- Provides guidelines for writing clear, testable requirements
- Used to structure the requirement definitions in Section 5.1

IEEE 1016 – Software Design Documentation

- Defines formats and conventions for documenting system architecture and flow
- Used for block diagrams, flowcharts, and unit designs

5.6.2 Coding Standards

Python coding guidelines were influenced by **PEP 8 – Python Style Guide**, which ensures readability and consistency.

Key conventions used:

- Meaningful variable and function names
- Consistent indentation (4 spaces)
- Logical modularization (functions → modules → system)
- Commenting and docstrings for clarity
- Avoiding redundant loops or nested conditions

Example:

```
def assign_faculty(course_list, faculty_data):  
    """Assigns faculty to each course based on availability."""  
    ...
```

5.6.3 Data Standards

Since the project handles student-related course data, certain data practices are required:

a) Data formatting standards

- CSV format for input files
- Dictionary or list-of-dictionaries for internal structures
- CSV/TXT export format for timetable outputs
- Encoding standard: UTF-8

b) Data validation standards

- No missing course codes
- Registration numbers must be integers
- Courses with fewer than 21 enrollments must be filtered
- MOOC list must be applied before conflict detection

c) Data security and privacy

While not regulated at the institutional level, the project voluntarily adopts:

ISO/IEC 27001 principles

- Access control to input files
- No storage of personal student identifiers
- Use of anonymized datasets for testing

5.6.4 Documentation Standards

Documentation follows:

IEEE 1063 – Documentation for Software Systems

Applies to:

- Introduction
- Methodology
- Analysis and design
- System requirements
- Flowcharts
- Testing results

IEEE Referencing Style

Used in the References section throughout the report.

5.6.5 Usability Standards

Although the project uses a console-based interface, the output must be readable and structured. The design follows:

ISO 9241 – Ergonomics of Human-System Interaction

Applied through:

- Clear, readable table formats
- Proper spacing between rows/columns
- Avoiding long unstructured text blocks
- Consistent naming conventions for slots, sections, faculty, and rooms

5.6.6 Output & Reporting Standards

To ensure consistent reporting:

- Tables follow **IEEE table formatting rules**
- Figures follow IEEE captioning rules (caption *below* image)
- Flowcharts follow standard process symbols
- Block diagrams follow functional block design rules

Suitability of Standards for This Project

The selected standards are suitable because:

- They ensure a structured software development process (ISO 12207)
- They maintain clarity in coding (PEP 8)
- They protect the integrity of student data (ISO 27001 principles)
- They ensure high-quality documentation (IEEE 1063)
- They produce usable and readable output (ISO 9241)

Together, these standards create a robust, maintainable, and user-friendly timetable generation system.

5.7 Other Design aspects

The *Other Design* section includes all remaining structural and architectural considerations needed to complete the Analysis & Design phase. Although the project is software-only and does not involve IoT, hardware, or communication protocols, several additional design elements contribute to system clarity, maintainability, and scalability. These include **data structure design, algorithmic design, file structure design, error-handling design, and output formatting design**.

5.7.1 Data Structure Design

Data structure selection is crucial for efficient conflict detection and greedy graph coloring. The following structures are used:

a) Dictionaries

Used extensively due to $O(1)$ access time.

- `course_students = {course: [students...]}`
- `conflict_graph = {course: set(conflicting_courses)}`
- `color_map = {course: assigned_color}`

b) Lists

- Storing student registration rows
- Maintaining ordered color groups
- Lists of sections for each course

Example:

`sections['CSE2013'] = ['CSE2013_A', 'CSE2013_B']`

c) Sets

Used for fast conflict checking due to $O(1)$ membership tests.

Example:

`students_in_both = set(courseA)  $\cap$  set(courseB)`

These data structures were selected to ensure low computational overhead during large dataset processing.

5.7.2 Algorithmic Design

The system uses the following algorithms:

a) Data Filtering Algorithm

- Remove MOOC courses
- Remove courses with <21 registrations
- Validate rows and empty entries

b) Conflict Graph Algorithm

- Compare enrolment lists
- Add edge if common students exist
- Construct adjacency set

c) Greedy Graph Coloring Algorithm

- Assign smallest possible color (slot)
- Ensures non-conflicting courses share slots
- Efficient for large graphs with sparse edges

d) Sectioning Algorithm

- If students > 60: split
- If between 40–60: optional split
- Ensures equal distribution

e) Resource Allocation Algorithm

- Faculty assigned using mapping priority
- Rooms assigned based on capacity

These designs ensure an efficient and scalable system suitable for real-time timetable generation.

5.7.3 Project File Structure Design

To keep the project organized, the following folder structure is used:

/Timetable_Project

```
| — data/  
|   | — registrations.txt  
|   | — faculty.csv  
|   | — rooms.csv  
|   | — mooc_list.txt  
| — src/  
|   | — preprocessing.py  
|   | — conflict_graph.py  
|   | — coloring.py
```

```
| |— allocation.py
| |— output_generator.py
|— output/
| |— final_timetable.csv
| |— logs.txt
|— docs/
| |— report_chapters.docx
```

This modular structure enhances maintainability and aligns with industry project organization standards.

5.7.4 Error Handling Design

To ensure robustness, the system includes:

- File not found checks
- Invalid format warnings
- Empty dataset detection
- Duplicate student ID removal
- Overlapping faculty schedule warnings
- Room capacity mismatch detection

Each error type triggers:

- **User-friendly message**
- **Logging in /output/logs.txt**
- **Graceful program continuation**

5.7.5 System Scalability Considerations

The design allows:

- Adding more time slots without restructuring code

- Handling larger datasets
- Adding more departments or electives
- Integrating web UI in future versions

The modular pipeline (preprocess → graph → coloring → allocation → output) supports easy expansion.

Chapter 6

Software and Simulation

6.1 Software development tools

The *Summer Term Timetable Generation System* was developed using **Google Colab**, a cloud-based Python execution environment. Colab provides an interactive notebook interface with built-in support for Python, making it highly suitable for data processing, algorithm development, and rapid prototyping.

6.1.1 Programming Language

- **Python 3.x (Colab default environment)**
 - Free, open-source
 - Pre-installed libraries
 - Ideal for algorithmic and data-driven systems

6.1.2 Development Environment

Google Colab (Primary Development Tool)

- Cloud-based Python notebook
- Does not require installation
- Provides free GPU/CPU resources
- Allows easy sharing and collaboration
- Built-in file upload, mount, and dataset handling
- Supports visualization and inline rendering

Reasons for using Google Colab:

- Accessible from any device
- No setup required
- Fast execution for Python scripts

- Easy integration with Google Drive for dataset storage
- Suitable for modular notebook-based development

6.1.3 Python Libraries Used

Table 6.1.3 Python Libraries

Library	Purpose
pandas	Data preprocessing (filtering, cleaning, merging)
collections	Dictionaries, default dicts for conflict graph
json/csv	Reading/writing structured files
matplotlib / seaborn (optional)	Data visualization
NumPy	Efficient numerical operations
NetworkX (optional)	Graph building and coloring visualization

All libraries were installed using Colab's built-in environment:

```
!pip install pandas numpy networkx
```

6.1.4 Version Control Tools

Even though Colab handles autosave, the project can be synced using:

- **Google Drive** (for datasets & notebook backups)
- **GitHub** (optional integration through Colab)

6.1.5 Documentation Tools

- **Google Docs / MS Word**
- **Draw.io** (for diagrams)
- **Colab Notebook Outputs** (for tables, visuals, logs)

6.2 Software Code

The *Summer Term Timetable Generation System* was implemented and executed entirely in **Google Colab**, which provides a browser-based Python environment with support for data

uploads, Drive integration, testing, and visualization. The code was developed in modular notebook cells following the unit design from Chapter 5.

6.2.1 Code Structure

Instead of multiple .py files, the project was structured into **notebook sections**, each implementing one module:

1. **Data Loading & Preprocessing**
2. **Conflict Graph Construction**
3. **Greedy Coloring**
4. **Section Allocation**
5. **Faculty & Room Assignment**
6. **Timetable Output Generation**
7. **Simulation & Visualization**

6.2.2 Example Code Snippets

a) Preprocessing and Filtering

```
def filter_courses(course_data, mooc_list, min_registration=21):  
  
    filtered = {}  
  
    for course, students in course_data.items():  
  
        if course not in mooc_list and len(students) >= min_registration:  
  
            filtered[course] = students  
  
    return filtered
```

b) Conflict Graph Construction

```
def build_conflict_graph(course_students):  
  
    graph = {course: set() for course in course_students}  
  
    for c1 in course_students:
```

```

for c2 in course_students:

    if c1 != c2:

        # Check common students

        if set(course_students[c1]) & set(course_students[c2]):

            graph[c1].add(c2)

            graph[c2].add(c1)

return graph

```

c) Greedy Coloring

```

def greedy_color(graph):

    color_map = {}

    for course in sorted(graph, key=lambda x: len(graph[x]), reverse=True):

        used = {color_map[n] for n in graph[course] if n in color_map}

        color = 1

        while color in used:

            color += 1

        color_map[course] = color

    return color_map

```

d) Section Allocation

```

def assign_sections(course_students):

    sections = {}

    for course, students in course_students.items():

        if len(students) > 60:

            mid = len(students)//2

            sections[course] = [students[:mid], students[mid:]]

```

```
else:
    sections[course] = [students]
```

```
return sections
```

e) Final Output Generator

```
def generate_timetable(color_map, faculty_map, room_map):
```

```
    result = []
```

```
    for course, slot in color_map.items():
```

```
        result.append({
```

```
            "course": course,
```

```
            "slot": slot,
```

```
            "faculty": faculty_map.get(course, "TBD"),
```

```
            "room": room_map.get(course, "TBD")
```

```
        })
```

```
    return result
```

6.2.3 Main Program

```
def main():
```

```
    course_data = load_courses()
```

```
    course_data = filter_courses(course_data, mooc_list)
```

```
    graph = build_conflict_graph(course_data)
```

```
    colors = greedy_color(graph)
```

```
    sections = assign_sections(course_data)
```

```
    faculty_map = assign_faculty(sections)
```

```
room_map = assign_rooms(sections)
```

```
final_timetable = generate_timetable(colors, faculty_map, room_map)
```

```
export_to_csv(final_timetable)
```

```
if __name__ == "__main__":
```

```
    main()
```

6.3 Simulation

Simulation involves running the software with **actual or test datasets** to validate the output timetable.

6.3.1 Simulation Inputs

Simulation uses:

- Student registration data
- Faculty availability
- MOOC list to remove unwanted courses
- Room capacity

6.3.2 Simulation Procedure

1. Load Input Data

The Python script imports the dataset (CSV/TXT).

2. Preprocessing Simulation

- Removes MOOC courses
- Filters courses <21 registrations

3. Graph Generation Simulation

- Visual inspection or plotting of conflict graph
- Check adjacency list correctness

4. Coloring Simulation

- Ensure no two adjacent nodes share same color
- Compare time-slot distribution

5. Sectioning Simulation

- Validate section splits for large courses

6. Faculty & Room Allocation Simulation

- Ensure assignments do not overlap
- Check capacity constraints

6.3.3 Evaluation Metrics

The simulation was evaluated based on:

- **Number of conflicts:** Should be zero
- **Slot utilization:** Ensures balanced distribution
- **Section correctness:** Sections properly split
- **Faculty & room validity:** No overlaps, capacity satisfied

6.3.4 Suitability of Simulation

The simulation process is suitable because:

It verifies correct functioning of each algorithm

It identifies errors before real deployment

It validates scalability for large course datasets

It ensures reliability of the generated timetable

Chapter 7

Evaluation and Results

Evaluation is a critical stage of the project where the performance, accuracy, and correctness of the *Summer Term Timetable Generation System* are verified. The system was tested in **Google Colab** using real or sample datasets to ensure that:

- All modules work as intended
- No conflicts appear in the final timetable
- Sections, faculty, and room assignments are correct
- Outputs are readable and useful for academic operations

The evaluation is divided into four parts: **Test Points**, **Test Plan**, **Test Results**, and **Insights**.

7.1 Test points

Test points refer to the **specific functional behaviours** of the system that need to be verified.

For this project, the following test points were identified:

1. Data Loading

- Correct reading of registration files
- Handling missing or incomplete rows
- MOOC list imported correctly

2. Data Preprocessing

- Courses with <21 registrations must be removed
- All MOOC courses must be excluded
- Duplicates must be cleaned
- Dataset remains consistent

3. Conflict Graph Construction

- Correct detection of overlapping students
- Accurate adjacency mapping
- No false-positive or missed conflicts

4. Graph Coloring / Slot Allocation

- No two conflicting courses should share the same slot
- Minimum number of slots is used
- Slot assignments are consistent

5. Section Generation

- Courses with enrollment >60 are correctly split
- Students are evenly divided among sections

6. Faculty Assignment

- No faculty is double-booked
- Every section is assigned a faculty member

7. Room Allocation

- Room capacity $\geq$ student count
- No overlapping room schedule

8. Output Generation

- Timetable exported correctly
- All columns (course, slot, section, faculty, room) are accurate
- Output readable and formatted

7.2 Test Plan

A structured **Test Plan** was created to validate each functional unit and integrated behaviour.

Table 7.2 Test Plan

Test Case	Description	Input	Expected Output	Status
TC1 – Load Data	Verify dataset is imported correctly	Registration CSV	DataFrame with valid rows	Pass
TC2 – MOOC Filtering	Test MOOC removal	Course list + MOOC list	No MOOC in output	Pass
TC3 – Registration Filter	Remove courses <21 students	Dataset with mixed counts	Only courses ≥ 21 remain	Pass
TC4 – Conflict Detection	Check graph edges	Student-course mapping	Correct adjacency list	Pass
TC5 – Colouring Validity	Check slot conflicts	Conflict graph	No adjacent nodes share color	Pass
TC6 – Section Formation	Divide high-enrol courses	Enrollment > 60	Two sections created	Pass
TC7 – Faculty Assignment	Check faculty conflicts	Section list + faculty list	No double-booking	Pass
TC8 – Room Allocation	Check room capacity	Room list + sections	Capacity $\geq$ section size	Pass
TC9 – Output Generation	Validate timetable export	Processed tables	CSV/txt output	Pass
TC10 – Error Handling	Check missing data handling	Incorrect CSV	Warning + continue	Pass

7.3 Test Results

Below is a summary of the results observed during testing.

1. Data Loading

- No missing file errors
- All CSV/TXT files imported correctly

2. Preprocessing Results

- All MOOC courses successfully removed

- All courses with <21 enrollments eliminated
- Data remained consistent with no broken mappings

3. Conflict Graph Results

- Graph constructed accurately with correct adjacency relationships
- Verified using printed adjacency lists and manual cross-checking

4. Slot Allocation Results

- Using greedy coloring, **no slot conflicts** occurred
- Slot distribution was balanced

5. Section Generation Results

- Large courses correctly split
- Balanced section sizes ensured
- No empty sections created

6. Faculty Assignment Results

- No faculty assigned to overlapping slots
- Valid mapping achieved for all sections

7. Room Allocation Results

- All room capacities satisfied
- No room assigned twice in the same slot

8. Final Timetable Output

The exported timetable included:

- Course code
- Section
- Slot
- Faculty

- Room

SUMMER-TERM TIMETABLE				
Day	9:00am-11:00am	11:00am-1:00pm	Lunch Break	2:00pm-4:00pm
Monday	ECE2007.A : F1, F2, R1	MAT1002.A : F13, F14, R1	Lunch Break	CSE2014.A : F25, F26, R1
	ECE2007.B : F3, F4, R2	MAT1002.B : F15, F16, R2		CSE2014.B : F27, F28, R2
	ECE2007.C : F5, F6, R3	MAT1002.C : F17, F18, R3		MAT2003.A : F29, F30, R3
	CSE3078.A : F7, F8, R4	CSE3082.A : F19, F20, R4		CV1008.A : F31, F32, R4
	CSE2010.A : F9, F10, R5	CSE2015.A : F21, F22, R5		DES1148.A : F33, F34, R5
Tuesday	CSE3150.A : F11, F12, R6	LAW1007.A : F99, F100, R5	Lunch Break	CSE2001.A : F51, F52, R1
	CSE2007.A : F35, F36, R1	CSE2009.A : F43, F44, R1		CSE2001.B : F53, F54, R2
	CSE2007.B : F37, F38, R2	CSE2009.B : F45, F46, R2		CSE2001.C : F55, F56, R3
	MEC1006.A : F39, F40, R3	MAT1003.A : F47, F48, R3		CSE3151.A : F57, F58, R4
	MEC1006.B : F41, F42, R4	PPS4006.A : F49, F50, R4		CSE1006.A : F59, F60, R5
	MGT2020.A : F65, F66, R1	CSE2018.A : F75, F76, R1	Lunch Break	CSE1006.B : F61, F62, R6
	MGT2020.A : F67, F68, R2	PPS4007.A : F77, F78, R2		MGT2021.A : F63, F64, R7
				CSE2074.A : F85, F86, R1
				PPS1001.A : F87, F88, R2

9. Error Handling Results

Tested with:

- Missing values
- Duplicates
- Incorrect file types
- Empty rows

System behaved as expected with warnings and stable output.

7.4 Insights

The evaluation process led to the following insights:

1. Greedy Coloring Works Efficiently

Despite being a simple algorithm, greedy coloring successfully generated **conflict-free** slots for all courses.

It is efficient and suitable for large datasets when used in a university timetable.

2. Preprocessing Quality Affects Accuracy

Errors in registration data can lead to incorrect conflict generation.

Thus, **data cleaning is crucial** for timetable accuracy.

3. Section Splitting Helps Reduce Slot Pressure

Splitting large courses into A/B sections:

- Reduced conflicts

- Balanced classroom occupancy
- Distributed faculty load more evenly

4. Faculty and Room Assignment Are Sensitive

Faculty or room shortages immediately cause scheduling issues.

Automated validation is essential to avoid double-bookings.

5. Google Colab Is Ideal for Academic Projects

Using Colab:

- Eliminated local system dependency
- Made testing fast and collaborative
- Simplified file upload, Drive storage, and visualization

6. System Scalability

The design is scalable:

- More slots can be added
- New departments or electives can be included
- Easy to extend to web-based UI

7. Zero-Conflict Output Achieved

All simulations confirmed:

- **No student conflicts**
- **No faculty conflicts**
- **No room conflicts**

This proves the **correctness and reliability** of the designed sys

Chapter 8

Social, Legal, Ethical, Sustainability and Safety aspects

Technology projects influence society in several ways, and responsible development requires consideration of social, legal, ethical, sustainability, and safety dimensions. Though the *Summer Term Timetable Generation System* is a non-IoT, non-hardware, software-based academic tool, it still interacts with sensitive academic data and influences educational workflows.

Therefore, the system must adhere to acceptable societal norms, legal requirements, ethical principles, and sustainable digital practices.

8.1 Social Aspects

Social aspects focus on how the timetable system impacts students, faculty, and the academic community.

Positive Social Impacts

1. **Reduced Student Stress**

A conflict-free timetable ensures that students do not face overlapping classes, reducing academic pressure and improving the learning experience.

2. **Improved Faculty Coordination**

Faculty members receive clear, non-conflicting schedules, facilitating better preparation and classroom management.

3. **Administrative Efficiency**

Automation reduces manual workload for academic coordinators, saving time and minimizing human errors.

4. **Equitable Access to Course Slots**

Fair slot allocation ensures that all students receive equal academic opportunities.

Negative or Potential Concerns

1. **Dependence on Software**

Over-reliance on automated scheduling can reduce human review if not monitored properly.

2. Digital Divide

Staff unfamiliar with computing tools may require training to utilize the system effectively.

8.2 Legal Aspects

Legal aspects cover compliance with regulations, data protection laws, and institutional policies.

Relevant Legal Considerations

1. Data Privacy

Student registration data is considered sensitive academic information.

Systems must follow principles similar to India's Digital Personal Data Protection Act (DPDPA 2023) and global standards such as the GDPR.

2. Data Minimisation Principle

Only essential information is used (course codes, student IDs—not personal info).

3. Lawful and Fair Processing

Data processing is conducted solely for timetable generation, respecting purpose limitation.

4. Institutional Compliance

Universities typically enforce internal rules regarding student information handling.

Legal Risks if Misused

- Unauthorized access to academic data
- Manipulation of schedules for unfair advantage
- Breach of confidentiality leading to disciplinary action

8.3 Ethical Aspects

Ethical considerations ensure fairness, transparency, and responsible use of the timetable system.

Ethical Responsibilities

1. Fairness in Slot Allocation

The algorithm must not privilege any department or group of students.

2. Accountability

Administrators must ensure the system is used honestly and not manipulated.

3. Transparency

The logic for conflict detection and slot allocation should be documented clearly.

4. Data Confidentiality

Student identities must remain protected during processing.

Potential Ethical Issues

- Misuse of data for favoritism
- Hidden manipulation of faculty or room assignment
- Over-automation reducing human oversight

Project Impact on Quality of Life

- Enhances academic scheduling transparency
- Reduces workload on faculty and staff
- Minimizes errors that can affect students' academic performance

8.4 Sustainability Aspects

Sustainability focuses on reducing resource usage and supporting long-term digital efficiency.

While this is a software-only project, sustainability still applies in areas such as:

1. Efficient Use of Digital Resources

- The system minimizes manual paperwork, reducing paper waste.
- Running the code in Google Colab optimizes cloud resources and avoids unnecessary hardware use.

2. Resource-Efficient Design

- The program avoids heavy computation or energy-intensive processes.
- Lightweight algorithms (e.g., greedy coloring) reduce CPU usage.

3. Durable and Maintainable Software Design

- Modular structure enables long-term reusability.
- Easy to update for future academic years without redesign.

4. High Digital Disposability

- All files are stored digitally, reducing physical waste.

5. Efficient Logistics

- Entire timetable distribution happens online (CSV/PDF).
- Zero transportation, zero printing cost.

6. Safe for Consumers

- No environmental risks since this is a digital-only tool.

8.5 Safety Aspects

Though the project is not IoT-based, safety still applies regarding **data safety, system reliability, and operational safety**.

1. Data Safety

- Academic data must be stored securely in Google Drive or institutional storage.
- Proper access control and avoiding sharing of sensitive datasets are essential.

2. Software Reliability

- Conflict-free timetables reduce the risk of academic mismanagement.
- System stability ensures classes run on time without disruptions.

3. Cybersecurity

- Files must not be shared publicly.
- Sensitive data must be anonymized before demos or publications.

4. Operational Safety

- The system avoids scheduling errors that can disrupt students' academic progress.

- Reliable timetable generation prevents accidental double-booking of rooms or faculty.

5. Prevention of System Misuse

- Access limited to authorized staff.
- Logs maintained for transparency.

Chapter 9

Conclusion

The *Summer Term Timetable Generation System* provides an effective and automated solution for generating a conflict-free academic timetable. By systematically processing student registration data, filtering courses, constructing a conflict graph, and applying a greedy graph-colouring algorithm, the system ensures that no two courses with common students are scheduled at the same time. The additional steps of splitting high-enrolment courses into sections and allocating faculty and rooms further help in producing a complete, practical, and usable timetable for academic planning.

The project successfully meets its intended objectives by reducing the manual workload on academic coordinators, minimizing scheduling errors, and improving the accuracy and reliability of the timetable. The automation of conflict detection and slot assignment helps streamline the scheduling process, while the structured output makes it easy for departments to review and finalize schedules. The system also demonstrates how data-driven methods can be applied effectively to academic administration tasks.

Overall, the results show that the approach used in this project is both efficient and scalable. In the future, the project can be enhanced by developing a user-friendly interface, enabling real-time updates, integrating multiple departments, or incorporating advanced optimization techniques to handle more complex scheduling scenarios. These improvements would make the system even more flexible and useful for institutional deployment.

References

- [1] Dasari, C.A., Kumar, S.M., Arpitalaxmi and Thushara, M.G., 2024. Optimizing University Time Table Schedules for Smart Space Utilization. *IEEE GCAT 2024*, Amrita Vishwa Vidyapeetham, India.
- [2] Khokale, S.R., Jadhav, A., Chavan, R., Wani, S. and Iwanate, P., 2024. A Survey Paper on Timetable Generator Using AI Methods. *IRJAEH Journal*, Guru Gobind Singh College, Nashik, India.
- [3] Kumar, S., et al., 2024. Adaptive Scheduler: AI Optimization of Academic Timetable. *IJS DR Journal 2024*, CMR Institute of Technology, Bengaluru, India.
- [4] Indian Student Researchers, 2024. Automatic Timetable Generator. *IARJSET Proceedings 2024*, Indian Engineering College Authors.
- [5] Indian Researchers, 2024. Automatic Timetable Generator using Genetic Algorithm. *Atlantis Press 2024*, Indian Academic Institutions.
- [6] Kumar, A. and Sharma, P., 2023. University Course Timetable Scheduling using Metaheuristic Algorithms. *IEEE ICACCS 2023*, IIT Delhi, India.
- [7] Teja, R. and Agarwal, N., 2025. AI-Driven Summer Term Timetable Generation for Higher Education. *IEEE ICCES 2025*, NIT Warangal, India.
- [8] Patil, S. and Deepika, R., 2024. Automatic Timetable Generation using Genetic Algorithm in Indian Universities. *IEEE ICECA 2024*, Visvesvaraya Technological University, India.
- [9] Karthik, M. and Verma, S., 2023. Smart Timetable Management System with Constraint Handling. *IEEE ICICCS 2023*, Anna University, India.
- [10] Ahmed, S., Rahman, M. and Chowdhury, T., 2021. Automated timetable generation using graph colouring algorithms. *International Journal of Computer Science and Information Security*, 19(4), pp.45–53.
- [11] Nandal, P., Satyawali, A., Sachdeva, D. and Tomar, A.S., 2021. *Graph coloring based scheduling algorithm to automatically generate college course timetable*. In *2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence)* (pp.210–214). IEEE.

- [12] Sahoo, R. and Mohanty, B., 2022. Conflict-free scheduling using graph theoretic techniques. *Journal of Applied Computing Research*, 5(1), pp.1–12.
- [13] Kumar, A. and Rout, N., 2020. Optimization of university course scheduling using greedy and heuristic techniques. *ICCIKE 2020*, pp.220–226.
- [14] Pressman, R.S. and Maxim, B.R., 2020. *Software Engineering: A Practitioner's Approach*. 9th ed. McGraw-Hill.
- [15] Sommerville, I., 2016. *Software Engineering*. 10th ed. Pearson Education.
- [16] IEEE Standards Association, 2017. *IEEE Std 1016-2017: Standard for Software Design Descriptions*. IEEE.
- [17] ISO/IEC, 2017. *ISO/IEC 12207: Systems and Software Engineering — Software Life Cycle Processes*. International Organization for Standardization.
- [18] United Nations, 2023. *Sustainable Development Goals (SDGs)*. UN Department of Economic and Social Affairs. Available at: <https://sdgs.un.org/goals>
- [19] Government of India, 2023. *Digital Personal Data Protection Act (DPDPA)*. Ministry of Electronics and Information Technology.
- [20] European Union, 2018. *General Data Protection Regulation (GDPR)*. Regulation (EU) 2016/679.
- [21] GeSI, 2021. *Digital with Purpose: Delivering a Smarter, More Sustainable World*. Global e-Sustainability Initiative.
- [22] Floridi, L., 2013. *The Ethics of Information*. Oxford University Press.

Base Paper

Google Scholar Automated timetable generation using graph colouring algorithms

Articles About 18,100 results (0.18 sec) My profile My library

Any time
 Since 2025
 Since 2024
 Since 2021
 Custom range...

Sort by relevance
 Sort by date

Any type
 Review articles

☐ include patents
☒ include citations
☒ Create alert

Graph coloring based scheduling algorithm to automatically generate college course timetable [PDF] researchgate.net
 P Nandal, A Satyawali, D Sachdeva... - ... Conference on Cloud ..., 2021 - ieeexplore.ieee.org
 ... Kolhe [15] uses genetic algorithms to create a semi-automatic timetable. They ... In this study graph coloring (vertex coloring) technique is used to solve the time table scheduling problem. ...
 ☆ Save Cite Cited by 14 Related articles All 2 versions

Timetable scheduling using graph coloring [PDF] academia.edu
 NK Cauvery - International Journal of P2P Network Trends and ..., 2011 - academia.edu
 ... Graph coloring has numerous applications in scheduling and ... approaches to solve graph coloring is "Ant algorithm" [1]. ... The automation of timetable problem is thus an important task as ...
 ☆ Save Cite Cited by 26 Related articles All 2 versions

Automated timetable generator for educational institutions using graph colouring technology [PDF] academia.edu
 S Gajbhiye, N Shende, U Sahu, Y Behar - IJAR, 2018 - academia.edu
 ... In this paper, Graph Colouring Algorithm is used to generate the weekly time table of the student for the college. We describe the software of the Graph Colouring Algorithm to a highly ...
 ☆ Save Cite Cited by 1 Related articles All 2 versions

Intelligent timetable scheduler: A comparison of genetic, graph coloring, heuristic and iterated local search algorithms [PDF] researchgate.net
 TW Ekanayake, P Subasinghe, S Ragel... - ... on Advancements in ..., 2019 - ieeexplore.ieee.org

×

Cite

MLA	Nandal, P., et al. "Graph coloring based scheduling algorithm to automatically generate college course timetable." <i>2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence)</i> . IEEE, 2021.
APA	Nandal, P., Satyawali, A., Sachdeva, D., & Tomar, A. S. (2021, January). Graph coloring based scheduling algorithm to automatically generate college course timetable. In <i>2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence)</i> (pp. 210-214). IEEE.
Chicago	Nandal, P., Ankit Satyawali, Dhananjay Sachdeva, and Abhinav Singh Tomar. "Graph coloring based scheduling algorithm to automatically generate college course timetable." In <i>2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence)</i> , pp. 210-214. IEEE, 2021.
Harvard	Nandal, P., Satyawali, A., Sachdeva, D. and Tomar, A.S., 2021, January. Graph coloring based scheduling algorithm to automatically generate college course timetable. In <i>2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence)</i> (pp. 210-214). IEEE.
Vancouver	Nandal P, Satyawali A, Sachdeva D, Tomar AS. Graph coloring based scheduling algorithm to automatically generate college course timetable. In <i>2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence)</i> 2021 Jan 28 (pp. 210-214). IEEE.

Appendix

Actual Roll Number	Student Name	Course Code
20191CCE0081	VENKAT VINAY	CSE2013
20191CCE0081	VENKAT VINAY	CSE2067
20191CCE0081	VENKAT VINAY	CSE240
20191CCE0081	VENKAT VINAY	CSE3150
20191CSE0048	B A SUMANTH JAIN	CSE220
20191CSE0048	B A SUMANTH JAIN	CSE248
20191CSE0048	B A SUMANTH JAIN	CSE257
20191CSE0048	B A SUMANTH JAIN	CSE307
20191CSE0422	PAWAN KUMAR P	CSE220
20191CSE0422	PAWAN KUMAR P	CSE256
20191CSE0422	PAWAN KUMAR P	CSE399
20191CSE0422	PAWAN KUMAR P	PPS108
20191CSE0545	SHAIK MOHAMMAD ADNAN SAMI	MEC152
20191CSE0545	SHAIK MOHAMMAD ADNAN SAMI	PIP101
20191ISE0109	MUJAHID UR REHMAN	CHE1001
20191ISE0109	MUJAHID UR REHMAN	CSE2013
20191ISE0109	MUJAHID UR REHMAN	CSE2054

The screenshot shows a Jupyter Notebook titled 'Untitled50.ipynb'. The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar with icons for commands, code, text, and running, and a sidebar with navigation icons. The main area displays a code cell with the following Python code:

```

def getTextLinesFromFile(fileName):
    ret = []
    with open(fileName, "r") as file:
        lines = file.readlines()
        for line in lines:
            ret.append(line.strip())
    return ret

lines = getTextLinesFromFile("dataset.txt")
print(len(lines))

```

The output of the code cell is 4230.

```

... FINAL TIMETABLE WITH UNIQUE FACULTY + ROOMS

Monday 9:00am-11:00am:
ECE2007-A : F1, F2 ; R1
ECE2007-B : F3, F4 ; R2
ECE2007-C : F5, F6 ; R3
CSE3078-A : F7, F8 ; R4
CSE2010-A : F9, F10 ; R5
CSE3150-A : F11, F12 ; R6

Monday 11:00am-1:00pm:
MAT1002-A : F13, F14 ; R1
MAT1002-B : F15, F16 ; R2
MAT1002-C : F17, F18 ; R3
CSE3082-A : F19, F20 ; R4
CSE2015-A : F21, F22 ; R5
LAW1007-A : F99, F100 ; R5

Monday 2:00pm-4:00pm:
CSE2014-A : F25, F26 ; R1
CSE2014-B : F27, F28 ; R2
MAT2003-A : F29, F30 ; R3

```

Keerthi D V

SUMMER-TERM TIMETABLE GENERATION

 Quick Submit
 Quick Submit
 Presidency University

Document Details

Submission ID
trmcoid::1:3425288780

Submission Date
Nov 26, 2025, 2:46 PM GMT+5:30

Download Date
Nov 26, 2025, 2:52 PM GMT+5:30

File Name
Project_Report_1_1_1.pdf

File Size
3.4 MB

82 Pages

12,378 Words

78,517 Characters



Page 1 of 87 - Cover Page

Submission ID trmcoid::1:3425288780



Page 2 of 87 - Integrity Overview

Submission ID trmcoid::1:3425288780

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

► Bibliography